

The Quantum Random Oracle Toolbox

Hans Heum 

Norwegian University of Science and Technology, Trondheim, Norway.
`hans.heum@ntnu.no`

Abstract. Proofs of security in the Quantum Random Oracle Model (QROM) face three main challenges, relating to the non-applicability of several conceptually simple yet powerful proof techniques that sit at the heart of its classical counterpart:

1. Recording: Or, how to learn from oracle queries.
2. Reprogramming: Or, how to alter the output on certain inputs.
3. Simulation: Or, how to efficiently implement a random oracle.

In this gentle introduction to the QROM, we will show how to recover these capabilities. Our approach will be to take a simple encryption scheme due to Bellare and Rogaway [BR93] as our motivating example, and prove increasingly strong statements about it, as we move through the literature on quantum random oracles and enlarge our toolbox.

This tutorial only assumes knowledge of basic linear algebra and standard game-hopping techniques (though some prior experience with the classical random oracle model doesn't hurt).

Keywords: Quantum Random Oracles · One-Way to Hiding · Compressed Oracle Technique

Table of Contents

1	Introduction	3
2	Preliminaries	4
3	What Are Quantum Random Oracles?	5
3.1	Classical Oracles in a Quantum World	6
3.2	Equivalent Implementations	6
3.3	Domain-Separating Quantum Random Oracles	7
4	How to Lazy-Sample a Quantum Random Oracle	9
4.1	The Setup	9
4.2	Defining CROs	10
4.3	Playing with CROs	11
4.4	Extracting Queries	15
	References	16
A	A Brief Quantum Primer	17
A.1	Dirac Notation	17
A.2	Qubit Basis States	17
A.3	General Qubit States	19
A.4	Qubit Registers	20
A.5	Unitary Operations	20
A.6	Quantum Entities, Entanglement, and Towards a Quantum Pseudocode	22
A.7	Formalizing Measurements	24
A.8	Select Exercise Solutions	28

1 Introduction

Proofs of security in the Random Oracle Model are often conceptually simple, and yet regularly achieve strong security claims with near-tightness for highly efficient constructions—which is as close as one gets to a holy trinity for provable security. In contrast, proofs in the Quantum Random Oracle Model are often hard to understand—even requiring specialized knowledge in quantum information—and infamously lossy.

And yet, if we want our proofs of security to have merit in a post-quantum world, it seems that our choice is between the Standard Model and the QROM. This is because random oracles model hash functions, which are publicly available algorithms that anyone can run locally on their machine. Therefore, there is nothing stopping someone with a quantum computer to boot it up and run it on a superposition of inputs. If we want to accurately model quantum attackers, then our model must include this capability.

In the standard model, of course, this is not a problem at all, since then the hash functions in question is provided to the attacker as-is. But if QROM proofs are lossy, then Standard Model constructions that achieve the same goals as their ROM counterparts are often highly inefficient in comparison—and, as every cryptographer knows, efficiency trumps security in real-world implementations.

Therefore, as we inch ever closer to the looming threat of cryptographically-relevant quantum computers, it is important that the barrier for entry into the Quantum Random Oracle Model is lowered as much as possible, so that the community may together bring our transition to a post-quantum world into effect, in a safe and (provably) secure manner.

The ROM techniques don't transfer . . . One of the most useful facts about reductions in the random oracle model, is that anyone can efficiently *simulate* a random oracle, simply by drawing the random outputs one-at-a-time, and storing them in a list to ensure consistency. This technique, known as *lazy sampling*, allows the reduction to not only implement the oracle efficiently, but also to see what input values the adversary is interested in, as well as adaptively *reprogram* the oracle on inputs of its choice. Indeed, much of the strength of the random oracle model lies in exactly these capabilities.

None of this transfers to the case where the oracle is quantum-accessible. To see why, consider an adversary that queries the oracle on a superposition of all possible inputs at once: The simulator would have to store an exponentially large list of input values in order just to process the input!

. . . But the concepts do. And yet, the past decade has seen the development of techniques that recover much of these capabilities. Prime among these are the various One-Way to Hiding (O2H) lemmas, that allow for careful reprogramming of the quantum random oracle, as well as limited query extraction; and the Compressed Random Oracle (CRO) technique, which allows a reduction to efficiently implement a simulation of a quantum random oracle, in direct correspondence to the “lazy sampling” technique described above.

Outline. After defining our notation in Sect. 2, we define quantum random oracles in Sect. 3. We provide several definitions, and show that they are all equivalent. In the process, we will try to get a “feel” for QROs, and we discuss how to (and how not to) reason intuitively about them.

In ??, we introduce our main motivating scheme, a hybrid public-key encryption scheme due to Bellare and Rogaway [BR93], and show that it achieves indistinguishability from the one-wayness of the underlying key encapsulation mechanism, by applying Unruh’s One-Way to Hiding lemma [?]. We then generalize this result several times, through application of newer and more general variations of the lemma, including the Semi-Classical O2H lemma due to Ambainis et al. [AHU19], the Adaptive Resampling lemma of Grilo et al. [GHHM21], the Adaptive Reprogramming framework of Pan and Zeng [PZ24], and the Fixed-Permutation framework of Jaeger [Jae24]. The goal of this exercise, of course, is *not* to show that the Bellare–Rogaway encryption scheme is secure, but rather to gain an understanding of the various O2H lemmas that exist in the literature, and when and how to apply each.

Then, in Sect. 4, we introduce the Compressed Random Oracle technique, which is to QROs as lazy sampling is to classical random oracles. This incredibly powerful technique takes some effort to wrap one’s head around, and so we dedicate significant space to give gentle and intuitive explanations for all its moving parts, as they are introduced.

Finally, with this technique in hand, we demonstrate its power by showing in ?? how we can use it to prove statements that were previously beyond our reach.

In addition, App. A contains a primer on quantum computing, including introductions to all the concepts and definitions for all the notation used in this text. It also includes several exercises to test your understanding, solutions to which are provided in App. A.8.

2 Preliminaries

Basic notation. We take the natural numbers $\mathbb{N}$ to include 0. For $n \in \mathbb{N}$, we define the set $[n] = \{0, \dots, n-1\}$. We will employ the convention that $N = 2^n$; thus, $NM = 2^{n+m}$. Functions are defined over bitstrings unless otherwise noted, and the set of all functions from the set of m -length bitstrings $\{0, 1\}^m$ to the set of n -length $\{0, 1\}^n$ bitstrings is denoted $\mathcal{F}_{m \rightarrow n}$. We will use $\mathbb{1}_n$ to denote the n -by- n identity matrix, dropping the subscript whenever the dimension is clear from context.

For a bitstring $x \in \{0, 1\}^*$, $|x|$ denotes its length. For a finite set $\mathcal{X}$, $|\mathcal{X}|$ is the cardinality of $\mathcal{X}$, and likewise for lists. For a list $\mathbf{X}$, $\mathbf{X}[i]$ retrieves the i th element of the list. Two lists $\mathbf{X}$ and $\mathbf{Y}$ may be combined by concatenation to form a new list $\mathbf{X} \parallel \mathbf{Y}$. We will at times treat bitstrings as lists of binary values.

We use $\Pr[\text{Code} : \text{Event} \mid \text{Condition}]$ to denote the conditional probability of **Event** occurring when **Code** is executed, conditioned on **Condition**. We omit **Code** when it is clear from context and **Condition** when it is not needed. Deterministic assignment is denoted $x \leftarrow y$, whereas $x \leftarrow \$ \mathcal{X}$ denotes uniform sampling from a set, and $x \leftarrow \$ Y(\cdot)$ conveys that the probabilistic algorithm Y halted with output x .

Game hopping. For our security proofs, we will be working in the game-hopping paradigm of Shoup [Sho04]. The fundamental lemma of game hopping states that the distinguishing advantage of an adversary can be bounded in terms of their distinguishing advantage over several intermediary game “hops”.

Lemma 1 (Fundamental Lemma of Game Hopping). *For any adversary $\mathbb{A}$ tasked with distinguishing G_0 from G_n , and for any games $G_1, \dots, G_{n-1}$, there exist adversaries $(\mathbb{A}_1, \dots, \mathbb{A}_n)$ such that*

$$|\Pr[G_0(\mathbb{A}) : 1 \leftarrow \$ \mathbb{A}] - \Pr[G_n(\mathbb{A}) : 1 \leftarrow \$ \mathbb{A}]| \leq \sum_{i=1}^n |\Pr[G_{i-1}(\mathbb{A}) : 1 \leftarrow \$ \mathbb{A}_i] - \Pr[G_i(\mathbb{A}) : 1 \leftarrow \$ \mathbb{A}_i]|.$$

The lemma states that, if $\mathbb{A}$ was able to distinguish, then at least one of $\mathbb{A}_1, \dots, \mathbb{A}_n$ must have been able to distinguish too—and vice versa, that if none of the $\mathbb{A}_i$ are able to distinguish, then neither is $\mathbb{A}$. In fact, the $\mathbb{A}_i$ that achieve this will essentially just be copies of $\mathbb{A}$.

The proof is remarkably simple, reproduced here for those that haven’t seen it.

Proof. Insert $0 = \Pr[G_i(\mathbb{A}_i) : 1 \leftarrow \$ \mathbb{A}_i] - \Pr[G_i(\mathbb{A}_i) : 1 \leftarrow \$ \mathbb{A}_i]$ into the left hand side for all i from 1 to $n-1$, and then apply the union bound. $\square$

Quantum notation. Precise definitions for our quantum-notational conventions are provided in App. A, along with a gentle introduction to the relevant quantum concepts, summarized in Table 1.

Table 1. An overview of our quantum notation. (lc. = lowercase, uc. = uppercase. *Not always well-defined.)

Object	Convention	Examples	Class/definition
Bitstring	Roman (lc., end of alph.)	x, y	$\in \{0, 1\}^n$
General state label	Greek (lc., middle of alph.)	ψ, φ	$\in \{\text{labels}\}$
Amplitude	Greek (lc, beginning of alph.)	α, β	$\in \mathbb{C}$
Qubit count	Roman (lc., middle of alph.)	n, m	$\in \mathbb{N}$
Dimensionality	Roman (uc., middle of alph.)	N, M	$:= 2^n, 2^m$
Computational basis state	Dirac ket w/ bitstring label	$ x\rangle$	$\in \mathbb{C}^M$
General quantum state	w/ General state label	$ \psi\rangle$	$\in \mathbb{C}^M$
Evolving state	w/ Time-dependent label	$ \psi(t)\rangle$	$\in \mathbb{C}^M, t \in \mathbb{N}$
Qubit	Lowercase q (sans serif)	q	Two-dim. physical system
Qubit register	Roman (uc., calligraphic)	$\mathcal{R}$	List of $m \leq n$ qubits $\mathbf{q}_i$
Global quantum system	Register $\mathcal{Q}$	$\mathcal{Q}$	List of n qubits $\mathbf{q}_i$
Qubit (pure) state*	Qubit label/index subscript	$ \mathbf{q}_i\rangle, \psi\rangle_i$	$\in \mathbb{C}^2$
Register (pure) state*	Register label/subscript	$ \mathcal{R}\rangle, \psi\rangle_{\mathcal{R}}$	$\in \mathbb{C}^M, m \leq n$
Global system state	Global system label	$ \mathcal{Q}\rangle$	$\in \mathbb{C}^N$
Unitary/measurement operators	Roman (uc., sans serif)	U, P, M	$\in \mathbb{C}^{M \times M}$
Apply unitary to register	Apply (unitary : register)	Apply (U : $\mathcal{R}$)	Method (see Def. 12)
Measure register	Meas (measurement : register)	$x \leftarrow \$ \mathbf{Meas}(\mathbf{M} : \mathcal{R})$	Method (see Def. 13)

3 What Are Quantum Random Oracles?

A classical oracle for a function f is a black-box procedure $\mathcal{O}_f$ such that, given input x , it returns $f(x)$ in such a way that no information about f , other than that which can be inferred from its output, is provided. It is helpful to think of the oracle as something that exists beyond your reach, “in the sky” so to speak, and with which you can only interact by sending inputs and receiving outputs. A random oracle, then, is simply a function oracle where the function f was chosen uniformly at random from the space of all functions from $\{0, 1\}^n$ to $\{0, 1\}^m$.

However, the operation $x \rightarrow f(x)$ is generally not reversible, and therefore not *unitary*, as is required of operations on quantum states. Therefore, we define *quantum-accessible* function oracles to be oracles that apply the unitary U_f , defined as the operation that takes computational basis states $|x, y\rangle$ to $|x, y \oplus f(x)\rangle$. Generally, the state of the input may be entangled with whoever is performing the query, so saying that the oracle takes as input a quantum *state*, e.g. $\sum_i \alpha_i |x_i, y_i\rangle$, would not be well-defined. Instead, we imagine that the query literally ships a collection of qubits over to the oracle, collected in the input and output registers $(\mathcal{X}, \mathcal{Y})$, and that the oracle then applies the U_f to the registers before shipping them back (see Fig. 1 for an illustration).¹

To see that this operation is unitary, it is enough to observe that it is *reversible*: Making a query with the same registers a second time yields the original state, as seen by its effect on the computational basis states:

$$|x, y\rangle \xrightarrow{\mathcal{O}_f} |x, y \oplus f(x)\rangle \xrightarrow{\mathcal{O}_f} |x, y \oplus f(x) \oplus f(x)\rangle = |x, y\rangle.$$

With this, we are ready to define quantum random oracles.

Definition 1 (Quantum Random Oracle). *An n -bit-to- m -bit Quantum Random Oracle (QRO), denoted H , is a quantum-accessible function oracle $\mathcal{O}_f$ for a uniformly random function $f \in \mathcal{F}_{n \rightarrow m}$.*

Remark 1. Classical random oracles typically follow the standard for hash functions in being defined from the space of *all* finite-length bit strings $\{0, 1\}^*$ to the space of fixed-length bitstrings $\{0, 1\}^m$. Technically, this means that f is not a function, but rather a collection of random functions f_ℓ , one for each input length ℓ . However, since quantum random oracles must operate by unitary means, we will make life easier for ourselves by explicitly restricting the input domain of the random function to be $\{0, 1\}^n$, and then just take for granted that any result trivially generalizes to collections of oracles with varying input length.

To see why this is the case, observe that, even when loading up a hash function on real-world quantum hardware, the input *length* to the hash function may not be placed in superposition, as this would not correspond to a well-defined unitary operation. Intuitively speaking, every “branch” in a superposition must be defined over the same number of qubits. Or in other words: Adding and removing qubits to an operation is a classical control mechanism, and not itself a quantum operation!

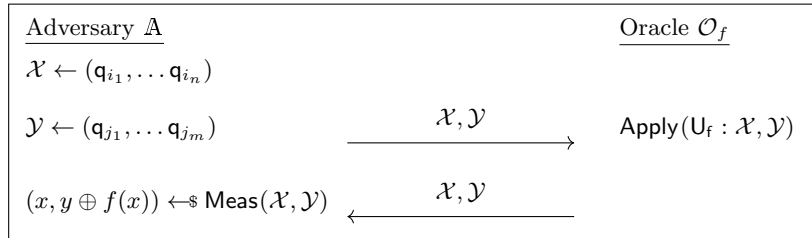


Fig. 1. An adversary A prepares the qubit registers $\mathcal{X}, \mathcal{Y}$, and sends them to a quantum-accessible function oracle $\mathcal{O}_f$, who applies unitary U to them and sends them back to the adversary. A then measures the registers to learn the values $(x, y \oplus f(x))$ corresponding to an element (x, y) in the initial superposition state of $\mathcal{X}, \mathcal{Y}$, chosen at random according to the distribution inferred by the quantum state.

¹ If any of this terminology confuses you, then this would be a good time to have a read through the quantum primer found in App. A!

Standard oracle $F(\mathcal{X}, \mathcal{Y})$	Phase oracle $G(\mathcal{X}, \mathcal{Y})$	Superposition oracle $H(\mathcal{X}, \mathcal{Y})$
if $\sigma = \varepsilon$: $f \leftarrow \$ \mathcal{F}_{n \rightarrow m}$ $\sigma \leftarrow f$ let U_f : $ x, y\rangle \rightarrow x, y \oplus f(x)\rangle$ Apply ($U_f : \mathcal{X}, \mathcal{Y}$) return $(\mathcal{X}, \mathcal{Y})$	if $\sigma = \varepsilon$: $g \leftarrow \$ \mathcal{F}_{n \rightarrow m}$ $\sigma \leftarrow g$ let U_g : $ x, y\rangle \rightarrow (-1)^{y \cdot g(x)} x, y\rangle$ Apply ($U_g : \mathcal{X}, \mathcal{Y}$) return $(\mathcal{X}, \mathcal{Y})$	if $\sigma = \varepsilon$: $ \mathcal{H}\rangle \leftarrow \sum_{\forall h \in \mathcal{F}_{n \rightarrow m}} h\rangle$ $\sigma \leftarrow \mathcal{H}$ let U_h : $ x, y\rangle h\rangle \rightarrow x, y \oplus h(x)\rangle h\rangle$ Apply ($U_h : \mathcal{X}, \mathcal{Y}, \mathcal{H}$) return $(\mathcal{X}, \mathcal{Y})$

Fig. 2. Various equivalent implementations of quantum random oracles. Here, σ is a state internal to the oracle, assumed initialized to the empty string ε , and $\mathcal{H}$ is a $(2^n \cdot m)$ -length qubit register.

3.1 Classical Oracles in a Quantum World

Sometimes, we want to provide a quantum algorithm with access to a classical oracle, but being unitary, we still face the issue of the operation $x \rightarrow f(x)$ not being unitary. The way to fix, and make classical oracles quantum-compatible, is to simply define classical oracles to be quantum oracles that first measure the input register, collapsing it to some basis state $|x\rangle$.

Definition 2 (Quantum-Compatible Classicle Oracle). *A quantum-compatible classical oracle is a function oracle for f that, on input qubit registers $(\mathcal{X}, \mathcal{Y})$, first performs the computational-basis measurement $\text{Meas}(\mathcal{X})$, and then performs the operation $\text{Apply}(U_f : \mathcal{X}, \mathcal{Y})$, before returning the registers to the sender. Here, f is the function defining the output of the classical oracle on input x , given any state and/or randomness internal to the oracle.*

Note that there is no need to collapse the output register $\mathcal{Y}$: If the input state is some superposition $\sum_i \alpha_i |x_i, y_i\rangle$, then after measuring the $\mathcal{X}$ register, it will have collapsed to the state $|x\rangle (\sum_i \beta_i |y_i\rangle)$ for some x , and some set of amplitudes $\{\beta_i\}$. Then the output state will be $|x\rangle (\sum_i \beta_i |y_i \oplus f(x)\rangle)$. Observe that the function f is still only evaluated on a single input, as desired.

With this definition in hand, we will go on to ignore these implementation details and write classical oracles, such as those appearing in our security games, in terms of their classical input/output interface.

3.2 Equivalent Implementations

Random oracles are typically treated as fully black-box, or “monolithic”, meaning that we don’t assume anything about their inner workings other than that it satisfies Def. 1. Still, it is instructive to see how a quantum random oracle *could* be implemented. Following are several possible implementations, which turn out to all be equivalent [Zha19].

Standard oracles. This “standard” oracle, shown to the left in Fig. 2, is the oracle that simply implements our intuition from the start of the section: It first draws a function f from $\{0, 1\}^n$ to $\{0, 1\}^m$ uniformly at random, storing the whole truth table as one exponential-sized list in the internal state σ , and thereafter operates as the quantum-accessible function oracle $\mathcal{O}_f$.

Phase oracles. The “phase” oracle, shown in the middle of Fig. 2, operates exactly like the standard oracle, except that it performs a phase operation on the output register rather than an xor. It therefore provides a different interface to the adversary, and is in that sense not equivalent in the sense of being indistinguishable, but *are* easily reducible to each other, as one kind of oracle becomes identical to the other if you just apply the Hadamard transformation $H^{\otimes m}$ to the output register $\mathcal{Y}$ before and after each query.

Superposition oracles. Instead of using a classically stochastic process to sample the random function at the outset, one can exploit the randomness inherent to quantum physics and simply provide a uniform superposition of *all* functions from $\{0, 1\}^n$ to $\{0, 1\}^m$. This “superposition” oracle, shown to the right in Fig. 2, is just as (in)efficiently implemented, and is completely indistinguishable from a standard oracle from the point of view of the adversary.

The reason, in a nutshell, is that when applying the unitary U_h , the oracle *entangles* the adversary with the gigantic superposition that it keeps. Hiding away entanglement in this manner has the effect, from the “local” point of view of the adversary, of introducing randomness into the state of $\mathcal{Y}$. In technical terms, the state will no longer be “pure” (but rather “mixed”), which could either be because of some hidden entanglement somewhere, or due to interference with a classically stochastic process—or both! However, there is no way to tell these cases apart without a full view of the combined system.

Of course, with the choice of interface being orthogonal to the choice of sampling vs. superposition, there is also a “superposition phase oracle”. We leave it to you to work out the details.

Compressed oracles. It turns out that there is a procedure through which we can efficiently implement a superposition oracle, as described above, in a manner completely analogous to the ubiquitous “lazy-sampling” technique for classical random oracles. Introduced by Zhandry [Zha19], these “compressed oracles” are enabled by the crucial insight that a uniform superposition over possible outputs will simply be represented by the all-zero string after a transformation to the Hadamard domain. Thus, we only have to store the output values for inputs that are *not* in a uniform superposition, due to having been entangled to the adversary via a query, leading to an efficient simulation procedure.

Of course, the above is an oversimplification: We still haven’t said anything about how to deal with large superpositions over inputs, for example. To get there, and to see that what we end up with really is a faithful simulation, we will first need to lay some more groundwork. Sect. 4 is dedicated to this.

3.3 Domain-Separating Quantum Random Oracles

Sometimes, a construction uses more than one random oracle, which are then required to be independent. A standard way to solve this in practice is via *domain-separation* [BDG20]: Given a random oracle H from $\{0,1\}^{n+1}$ to $\{0,1\}^m$ (or a hash function like SHA3, if you are in the real world), let the random oracle F from $\{0,1\}^n$ to $\{0,1\}^m$ be defined as $F(x) := H(0\|x)$, and likewise let the random oracle $G(x)$ be given by $G(x) := H(1\|x)$. Clearly, this works in the classical case, since the output of H is uniformly random on each input, so flipping even a single bit makes the outputs completely uncorrelated. Further, it should be clear that the other direction also holds: You don’t get anything *more* with oracle access to H directly than you do given oracle access to F and G separately.

But what about the quantum case? Now, we are supposed to be able to call the oracles in superposition, yet we can only call one oracle at a time. In particular, it seems that we can’t make a query that is “in a superposition of going to F and to G ”, even though this would be a trivial task given oracle access to H . Is it conceivable that we can do *more* if we are given oracle access to H instead?

The following lemma answers this question in the negative: Domain-separation works just as well in the QROM as it does in the classical ROM—thankfully!²

Lemma 2 (Two Oracles = One Oracle). *Let F and G be quantum random oracles from $\{0,1\}^n$ to $\{0,1\}^m$, and let H be a quantum random oracle from $\{0,1\}^{n+1}$ to $\{0,1\}^m$. Then, for any adversary $\mathbb{A}$ that expects simultaneous oracle access to F and G , there is a reduction $\mathbb{B}$ that, given oracle access to H , provides $\mathbb{A}$ with a perfect simulation of F and G .*

And vice versa: For any adversary $\mathbb{A}$ that expects oracle access to H , there is a reduction $\mathbb{B}$ that, given simultaneous access to F and G , provides $\mathbb{A}$ with a perfect simulation of H .

Proof. The first direction is trivial, and works exactly like the classical case: Whenever $\mathbb{A}$ makes a call to F with registers $(\mathcal{X}, \mathcal{Y})$, $\mathbb{B}$ makes an oracle call to H with registers $(q\|\mathcal{X}, \mathcal{Y})$, where q is a qubit placed in the state $|q\rangle = |0\rangle$; and likewise for calls to G , except with $|q\rangle$ set to $|1\rangle$. The registers $(\mathcal{X}, \mathcal{Y})$ are then returned to $\mathbb{A}$ as usual.

For the other direction, let the controlled swap gate **CSWAP** be defined by the following operation on computational basis states (where $|x\rangle = |z\rangle$):

$$|0\rangle|x\rangle|z\rangle \xrightarrow{\text{CSWAP}} |0\rangle|x\rangle|z\rangle, \quad |1\rangle|x\rangle|z\rangle \xrightarrow{\text{CSWAP}} |0\rangle|z\rangle|x\rangle.$$

The reduction $\mathbb{B}$ sets up up a new n -qubit register $\mathcal{Z}$, initialized to the uniform superposition over all $z \in \{0,1\}^n$. Then, on input a query to H with the registers $(q\|\mathcal{X}, \mathcal{Y})$, $\mathbb{B}$ does the following:

1. Apply **CSWAP** on $\mathcal{X}$ and $\mathcal{Z}$, with qubit q as the control;
2. Send $(\mathcal{X}, \mathcal{Y})$ to $\mathcal{O}[F]$, and $(\mathcal{Z}, \mathcal{Y})$ to $\mathcal{O}[G]$;

² I’m grateful to Joseph Jaeger for showing me this reduction.

3. Apply CSWAP a second time to the same registers;
4. Return $(\mathcal{X}, \mathcal{Y})$ to $\mathbb{A}$.

To see that this leads to the correct distribution, consider the result of applying this procedure to the first query if qubit q is in some superposition state $|q\rangle = \alpha|0\rangle + \beta|1\rangle$, and $\mathcal{X}$ and $\mathcal{Y}$ are set to the computational basis states $|x\rangle_{\mathcal{X}} |y\rangle_{\mathcal{Y}}$. Then we start out with the combined state

$$\begin{aligned} & (\alpha|0\rangle_q + \beta|1\rangle_q) |x\rangle_{\mathcal{X}} |y\rangle_{\mathcal{Y}} \left(\sum_{\forall z \in \{0,1\}^n} \frac{1}{\sqrt{2^n}} |z\rangle_{\mathcal{Z}} \right) \\ &= \frac{1}{\sqrt{2^n}} \sum_{\forall z \in \{0,1\}^n} \alpha|0\rangle_q |x\rangle_{\mathcal{X}} |y\rangle_{\mathcal{Y}} |z\rangle_{\mathcal{Z}} + \beta|1\rangle_q |x\rangle_{\mathcal{X}} |y\rangle_{\mathcal{Y}} |z\rangle_{\mathcal{Z}} . \end{aligned}$$

After applying the first CSWAP, we get the state

$$\xrightarrow{1.} \frac{1}{\sqrt{2^n}} \sum_{\forall z \in \{0,1\}^n} \left(\alpha|0\rangle_q |x\rangle_{\mathcal{X}} |y\rangle_{\mathcal{Y}} |z\rangle_{\mathcal{Z}} + \beta|1\rangle_q |z\rangle_{\mathcal{X}} |y\rangle_{\mathcal{Y}} |x\rangle_{\mathcal{Z}} \right) .$$

At this point, the registers $(\mathcal{X}, \mathcal{Y})$ are sent to F , and then registers $(\mathcal{Z}, \mathcal{Y})$ are sent to G , with the resulting state

$$\xrightarrow{2.} \frac{1}{\sqrt{2^n}} \sum_{\forall z \in \{0,1\}^n} \left(\alpha|0\rangle_q |x\rangle_{\mathcal{X}} |y \oplus F(x) \oplus G(z)\rangle_{\mathcal{Y}} |z\rangle_{\mathcal{Z}} + \beta|1\rangle_q |z\rangle_{\mathcal{X}} |y \oplus F(z) \oplus G(x)\rangle_{\mathcal{Y}} |x\rangle_{\mathcal{Z}} \right) .$$

Applying CSWAP a second time gives us

$$\xrightarrow{3.} \frac{1}{\sqrt{2^n}} \sum_{\forall z \in \{0,1\}^n} \left(\alpha|0\rangle_q |x\rangle_{\mathcal{X}} |y \oplus F(x) \oplus G(z)\rangle_{\mathcal{Y}} |z\rangle_{\mathcal{Z}} + \beta|1\rangle_q |x\rangle_{\mathcal{X}} |y \oplus F(z) \oplus G(x)\rangle_{\mathcal{Y}} |z\rangle_{\mathcal{Z}} \right) ,$$

at which point the registers $\mathcal{X}$ and $\mathcal{Y}$ are returned to $\mathbb{A}$.

Note that register $\mathcal{Y}$ ends up entangled with register $\mathcal{Z}$. As explained when discussing superposition oracles, hidden entanglement induces a classical probability distribution when it is hidden from view, and maximal entanglement with a hidden uniform superposition is indistinguishable from the uniform probability distribution. Since the values of F and G are xored into the output state, this uniformity carries over; and yet, again thanks to entanglement, we see that we are guaranteed consistency upon repeat queries.

Since the correct behaviour is exhibited on all input basis states $|b\rangle|x, y\rangle$, thanks to the linearity of quantum mechanics, we may conclude that the simulation exhibits the correct behaviour on all input states. $\square$

4 How to Lazy-Sample a Quantum Random Oracle

We will employ the compressed oracle technique, as introduced by Zhandry [Zha19]; specifically, we employ an adapted version of the “standard Fourier oracle”, the choices being standard/phase and computational/Fourier, depending on what oracle interface we want, and in which domain the database entries should be defined.³

4.1 The Setup

Let us begin by specifying our notation. To start with, unless noted otherwise, we will assume that all random oracles have input length n and output length m , and that q is an upper bound on the total number of queries made to the oracle.

The empty string. Let ε to be the empty string, encoded as $1\|0^n$. We will interpret the string $0\|x$ as x . Strings of the form $1\|x$ for $x \neq 0^n$ will never appear, but for the sake of reference let us call them $\not x$.

We define the $(1+n)$ -bit controlled-not operation on ε to be $\text{CX}_\varepsilon(|x\rangle_{\mathcal{A}} \rightarrow |\varepsilon\rangle_{\mathcal{B}}) = |x\rangle_{\mathcal{A}} |x\rangle_{\mathcal{B}}$. This may be implemented by performing the controlled not-operation as usual, and then flipping the leading bit of register $\mathcal{B}$. Repeating the operation yields $\text{CX}_\varepsilon(|x\rangle_{\mathcal{A}} \rightarrow |x\rangle_{\mathcal{B}}) = |x\rangle_{\mathcal{A}} |\varepsilon\rangle_{\mathcal{B}}$ again, making CX_ε an involution. (Applying CX_ε to states $|x\rangle |y\rangle$ such that $x \neq y$ will lead to the invalid state $|x\rangle |\not x\rangle$, where $z = x \oplus y$, but we will never apply the gate to such cases.)

Finally, observe that $x \cdot \varepsilon = 0$ for any x , where $\cdot$ denotes the bitwise inner product.

Databases. We are going to keep a database register $\mathcal{D}$, storing q list entries of the format $(b_\varepsilon\|x, \hat{y})$, meaning $\mathcal{D}$ is a register containing $q \cdot (1+n+m)$ qubits. The database starts out in the “empty” state, with each entry initialized in the pure state $|\varepsilon, 0^m\rangle$. After several queries, the database will end up in a superposition over lists $\mathcal{D}$, with each list being split in two parts: An “active” part of tuples $(x, \hat{y})$ such that $\hat{y} \neq 0^n$, sorted by increasing x , followed by “fresh” part of entries set to the initial value $(\varepsilon, 0^m)$. More precisely, after ℓ queries, $\mathcal{D}$ will be in a superposition over basis states of the form

$$\begin{aligned} |\mathcal{D}\rangle &= |(x_1, \hat{y}_1), (x_2, \hat{y}_2) \dots, (x_k, \hat{y}_k), (\varepsilon, 0^m), \dots, (\varepsilon, 0^m)\rangle \\ &= |x_1, \hat{y}_1\rangle_1 |x_2, \hat{y}_2\rangle_2 \dots |x_k, \hat{y}_k\rangle_k |\varepsilon, 0^m\rangle_{k+1} \dots |\varepsilon, 0^m\rangle_q, \end{aligned}$$

with unique and increasing input elements $x_1 < x_2 < \dots < x_k$, non-zero output elements $\hat{y}_i$, and $k \leq \ell$. We write $\mathcal{D}[x] = \hat{y}$ if the list contains an element $(x, \hat{y})$, and $\mathcal{D}[x] = \perp$ otherwise.

The state of the database register will in general be highly entangled with the parties performing the queries; indeed, a list element will be non-empty *only if* the register is entangled with the state of a querying party. Finally, the number of “active” versus “fresh” entries may differ between branches of the superposition of $\mathcal{D}$, though their sum will always be q .⁴

Database subregisters. Next, let us define a naming convention for the subregisters of $\mathcal{D}$: Let $\mathcal{D}_{\mathcal{X}}$ refer to the collection of qubits that store input values in the list (i.e. a subregister of size $q \cdot (1+n)$), and let $\mathcal{D}_{\mathcal{Y}}$ refer to the collection of qubits that stores output values (with size $q \cdot m$).

Now, provided $\mathcal{D}[x] \neq \varepsilon$, let $\mathcal{D}(x)$ refer to the qubits that store the relevant tuple $(x, \hat{y})$. Note that even in branches where this holds, the exact qubit indices corresponding to $\mathcal{D}(x)$ will in general differ between branches, since the lists in the superposition may contain a varying number of entries that precede x , making $\mathcal{D}(x)$ only well-defined when specializing to single branches of $\mathcal{D}_{\mathcal{X}}$ superposition. When speaking generally, we may use $\mathcal{D}_i$ to refer to the registers that will store the i ’th tuple of any list.

Combining, we can then refer to the subregister storing the first element of the tuple $(x, \hat{y})$ as $\mathcal{D}_{\mathcal{X}}(x)$, and the register storing the corresponding value $\hat{y}$ as $\mathcal{D}_{\mathcal{Y}}(x)$, (and likewise for $\mathcal{D}_{\mathcal{X},i}$, $\mathcal{D}_{\mathcal{Y},i}$).

³ The standard Fourier oracle is not the version employed by Zhandry in his paper, and it is only defined in an appendix, but it is the version I find most closely matches the intuition behind the compressed oracle technique.

⁴ In the original formulation of the compressed oracle technique, the size of $\mathcal{D}$ was allowed to grow and shrink with each call. This leads to a superposition over databases where also the *size* of the register is in superposition. In our formalization, registers are classically well-defined lists storing qubits (which are two-level physical systems), and while it is possible to resize registers by moving qubits around, we do not allow for this to happen in superposition. Therefore, we fix the size of $\mathcal{D}$ to be q a priori, and simply accept that the simulation will fail if the query count ever exceeds q .

Table 2. An overview of our notation for compressed random oracles.

Object	Notation	Class/definition
Input register	$\mathcal{X}$	n -length qubit register
Output register	$\mathcal{Y}$	m -length qubit register
Database	$\mathcal{D}$	$(q \cdot (1 + n + m))$ -length register
Empty string	ε	$:= 0\ 0^n$
Interpreted input string	x	$:= 0\ x, x \in \{0, 1\}^n$
Invalid string	$\nexists$	$:= 1\ x, x \in \{0, 1\}^n \setminus \{0^n\}$
Hadamard-domain value	$\hat{y}$	Labels basis states in the support of $H^{\otimes m} y\rangle$
List	$\mathbf{D}$	List of q entries
List entry	$(x, \hat{y})$	$(1 + n, m)$ -sized string tuple $(b_\varepsilon \ x, \hat{y})$
Initialized entry	$(\varepsilon, 0^m)$	$(1 + n, m)$ -sized string tuple
List contains $(x, \hat{y})$	$\mathbf{D}[x] = \hat{y}$	$x \in \{0, 1\}^n, \hat{y} = \{0, 1\}^m$
x is not in list	$\mathbf{D}[x] = \perp$	Special symbol denoting “undefined”
Input subregister	$\mathcal{D}_{\mathcal{X}}$	$(q \cdot (1 + n))$ -sized subregister storing input values
Output subregister	$\mathcal{D}_{\mathcal{Y}}$	$(q \cdot m)$ -sized subregister storing output values
Entry subregister	$\mathcal{D}[i]$	$(1 + n + m)$ -sized subregister storing i th entry
Input-specific subregister	$\mathcal{D}(x)$	Subregister $\mathcal{D}[i]$ storing $(x, \mathbf{D}[x])$ in current branch
Entry I/O registers	$\mathcal{D}_{\mathcal{X}}[i], \mathcal{D}_{\mathcal{Y}}[i]$	Subregisters storing i 'th input resp. output value
Specific I/O registers	$\mathcal{D}_{\mathcal{X}}(x), \mathcal{D}_{\mathcal{Y}}(x)$	Subregisters storing x resp. $\mathbf{D}[x]$ in current branch

The primal and the Hadamard domain. You may have noticed the hats. These denote that the value in question is defined in the Hadamard-domain: A computational basis state $|y\rangle$ in the primal domain (left) will be the following superposition in the Hadamard domain (right):

$$|y\rangle \xleftrightarrow{H^{\otimes m}} \sum_{\forall \hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |\hat{y}\rangle,$$

and vice versa:

$$\sum_{\forall y \in [2^m]} \frac{(-1)^{\hat{y} \cdot y}}{\sqrt{2^m}} |y\rangle \xleftrightarrow{H^{\otimes m}} |\hat{y}\rangle.$$

In particular, we have that the all-zero state in the Hadamard domain corresponds to the uniform superposition in the primal domain:

$$\sum_{\forall y \in [2^m]} \frac{1}{\sqrt{2^m}} |y\rangle \xleftrightarrow{H^{\otimes m}} |0^m\rangle.$$

As illustrated, one can move between the primal domain and the Hadamard domain by performing the “Hadamard transformation”, which is just applying the H gate to each qubit in the register individually.⁵ We will only move between domains on the output register, i.e. the $\mathcal{Y}$ register forwarded as part of the query. The input registers $\mathcal{X}$ and $\mathcal{D}_{\mathcal{X}}$ will both be kept in the primal domain, and the output register $\mathcal{D}_{\mathcal{Y}}$ of the database will be kept in the Hadamard domain.

4.2 Defining CROs

Updating the database. Next, we define a unitary that acts on the combined state of the input register $\mathcal{X}$ and the database $\mathcal{D}$, named S for “Store/Sort”.⁶ In essence, it is an operation that reads the input, and updates the database accordingly. We define it by specifying it on all basis states, verifying along the way that each step is reversible, and therefore unitary. Indeed, S will turn out to be an involution, meaning that applying it twice in a row will return the same state, (or in other words, that $S^2 = \mathbb{1}$).

⁵ In his paper [Zha19], Zhandry varyingly refers to the Quantum Fourier Transformation and the Hadamard transformation, but he clarifies in App. A that he only ever uses the latter. (The Hadamard gate H is the single-qubit Quantum Fourier Transformation.)

⁶ Zhandry calls this unitary **FourDecomp**, for “Fourier Decompress” [Zha19, App. C].

Definition 3 (Unitary S). For each computational basis state $|x\rangle |D\rangle$ in the support of the combined state of registers $\mathcal{X}$ and $\mathcal{D}$:

1. Do one of the following, depending on the case:
 - (a) If $D[x] = \hat{y} \neq 0^m$, then $H(x)$ has already been set, and $D(x)$ must furthermore be entangled with a querying party; do nothing.
 - (b) If $D[x] = \perp$, set the first empty element $(\varepsilon, 0^m)$ in D to $(x, 0^m)$ by applying CX_ε from $\mathcal{X}$ to the corresponding subregister $\mathcal{D}_{\mathcal{X},i}$.
 - (c) If $D[x] = 0^m$, “forget” the element by applying CX_ε from $\mathcal{X}$ to the subregister of $\mathcal{D}_{\mathcal{X}}(x)$, resulting in the entry $(\varepsilon, 0^m)$.
 - (d) For all other basis states, which is to say for lists D that contain no empty entries $(\varepsilon, 0^m)$ (which can only happen if the query count exceeds q), and for lists that contain invalid values $\hat{x}$ for some $x \neq 0^n$ (which will never happen), act as the identity.
2. Use controlled SWAP gates to sort D by increasing x .

The If-statements may be implemented as multi-bit controlled operations. Notice that the cases are mutually exclusive as well as exhaustive, and that they define an involution. Therefore, S is unitary.

Entangling the query. With the updating procedure defined, all that remains is to define how an output is provided.

In essence, the idea is to move the output register $\mathcal{Y}$ to the Hadamard domain, entangle it with the subregisters $\mathcal{D}_{\mathcal{Y}}(x)$ for each x in the input superposition, and then return $\mathcal{Y}$ to the primal domain, before shipping $\mathcal{X}$ and $\mathcal{Y}$ back to the querying party. The entanglement, which is hidden from view of whoever made the query, will make it look like a classical probability distribution has been induced over the possible states of $\mathcal{Y}$ —just as if H had been sampled from the set of all functions.

Let us define the “Entangling” unitary E next.

Definition 4. Let E be the unitary that acts on basis states $|x\rangle |y\rangle |D\rangle$ of registers $\mathcal{X}, \mathcal{Y}, \mathcal{D}$ in the following way:

$$|x\rangle |\hat{y}\rangle |\dots, (x, z), \dots\rangle \xrightarrow{E} |x\rangle |\hat{y}\rangle |\dots, (x, \hat{y} \oplus z), \dots\rangle,$$

and as the identity whenever $D[x] = \perp$.⁷ The cases are mutually exclusive and exhaustive, and define an involution. Therefore, E is unitary.

The oracle procedure. With S and E defined, we are finally ready to define the action performed by our compressed random oracle upon receiving a query.

Definition 5 (Compressed random oracle). Given registers $\mathcal{X}, \mathcal{Y}, \mathcal{D}$, the compressed random oracle $H : \{0, 1\}^n \rightarrow \{0, 1\}^m$ performs the following operations:

1. Apply($S : \mathcal{X}, \mathcal{D}$)
2. Apply($H^{\otimes m} : \mathcal{Y}$)
3. Apply($E : \mathcal{X}, \mathcal{Y}, \mathcal{D}$)
4. Apply($H^{\otimes m} : \mathcal{Y}$)
5. Apply($S : \mathcal{X}, \mathcal{D}$)

Having defined this procedure, Zhandry then proved that it provides a perfect simulation of a quantum-accessible random oracle [Zha19, Lemma 4].

Theorem 1 (Simulation is perfect). Let H be a quantum-accessible random oracle, and let G be a compressed random oracle. Then we have for all distinguishers $\mathbb{D}$,

$$\Pr[1 \leftarrow \$ \mathbb{D}^H] = \Pr[1 \leftarrow \$ \mathbb{D}^G].$$

4.3 Playing with CROs

What is the effect of the operations listed in Def. 5? Let’s explore!

⁷ We will ensure that this never occurs.

Classical queries. For our first sanity check, consider the result of performing the first three steps on a classical query, i.e. with $\mathcal{X}$ and $\mathcal{Y}$ put in a computational basis state $|x\rangle|y\rangle$, to the initially-empty database $|\mathcal{D}_\varepsilon\rangle$:

$$\begin{aligned} 1. & |x\rangle|y\rangle|\mathcal{D}_\varepsilon\rangle := |x\rangle|y\rangle|(\varepsilon, 0^m), \dots, (\varepsilon, 0^m)\rangle \xrightarrow{S} |x\rangle|y\rangle|(x, 0^m), (\varepsilon, 0^m), \dots, (\varepsilon, 0^m)\rangle := |x\rangle|y\rangle|\mathcal{D}_x\rangle, \\ 2. & |x\rangle|y\rangle|\mathcal{D}_x\rangle \xrightarrow{H^{\otimes m}} |x\rangle\left(\sum_{\hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |\hat{y}\rangle\right)|\mathcal{D}_x\rangle, \\ 3. & |x\rangle\left(\sum_{\hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |\hat{y}\rangle\right)|(\varepsilon, 0^m), \dots\rangle \\ & \xrightarrow{E} |x\rangle\left(\sum_{\hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |\hat{y}\rangle|(x, \hat{y}), (\varepsilon, 0^m), \dots)\right) := \sum_{\hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |x\rangle|\hat{y}\rangle|\mathcal{D}_{x, \hat{y}}\rangle. \end{aligned}$$

At this point, $\mathcal{Y}$ is maximally entangled with the subregister of $\mathcal{D}$ housing $\mathcal{D}[x]$: We can see from the above that their combined state now has the form (ignoring amplitudes) $|\hat{y}_1\rangle|\hat{y}_1\rangle + |\hat{y}_2\rangle|\hat{y}_2\rangle + \dots + |\hat{y}_{2^m}\rangle|\hat{y}_{2^m}\rangle$. This may be a large superposition, but the outcome of observing the two registers is guaranteed to be equal, which is exactly what it means to be maximally entangled (in general, that observing one register fully determines the state of the other). Performing the Hadamard transformation on $\mathcal{Y}$ thereafter does nothing to change this⁸, but rather has the effect of moving their combined state from one maximally entangled state to another.

Finally, the second application of S does nothing, since the list is already sorted and there is nothing to remove. That is, with one exception: In the single branch of the wavefunction where $\hat{y} = 0^m$, x will be replaced with ε , which has the effect of “implicitly” defining the value of $\hat{y}$ in this branch.

At a glance, this may appear to be a problem, since if the querying party then by chance observes exactly this branch, and then queries the compressed oracle again on the same input, the database would not “know” that it was supposed give the same value back, leading to an inconsistency. However, Thm. 1 tells us that simulation is perfect, so *something* will have to ensure consistency in this case. We will return to the question of how at the end of this section, though I urge you to work it out yourself and discover for yourself how it happens. (Hint: Try specializing to the $m = 1$ case.)

And so, accepting the conundrum for now, we return to our maximally entangled state. The entanglement is hidden from view of the querying party, and hiding away entanglement in this manner has the effect of “mixing” the state of $\mathcal{Y}$: From their point of view, hidden entanglement is indistinguishable from a classical probability distribution over quantum states⁹; and maximal entanglement yields maximally mixed states, which corresponds to a uniform probability distribution over basis states.

Thus, the querying party has the experience of having the $\mathcal{Y}$ register placed in a uniformly random computational basis state—exactly as if $H(x)$ had been sampled at random and xored into the register!

Uniform superpositions. Next, what if $\mathcal{Y}$ is set to be in the uniform superposition, $\sum_y \sqrt{2^{-m}} |y\rangle$ (with the input value still set to the computational basis state $|x\rangle$)? Well, then *nothing* happens: After being moved to the Hadamard domain, the $\mathcal{Y}$ register will be in the state $|0^m\rangle$, and xoring into the database has no effect: The second Hadamard transformation will take it back to the uniform superposition, and then return it as-is. Note that this also matches the querying party’s expectations: Since the xor of any fixed string defines a permutation, xoring into the uniform superposition should yield the uniform superposition, regardless of the value $H(x)$ that is being xored. This means that, when interacting with a real random oracle, $H(x)$ will remain perfectly hidden from their view. This is matched in the compressed oracle by the second application of S removing the point x from the list and replacing it with ε , ensuring no entanglement (which would otherwise have been a noticeable inconsistency, since the output register would no longer be in a uniform superposition, but rather in some mixed state).

What about other superpositions, for example $\sum_y (-1)^{y \cdot z} / \sqrt{2^{-m}} |y\rangle$ for some $z \in \{0, 1\}^m$? Well, then the querying party *would* expect to be able to learn something about the output, by observing how the phases shift around. The database will likewise be updated in this case: In our example, moving $\mathcal{Y}$ to the Hadamard domain will put it in the computational basis state $|z\rangle$, which will E will then store into the database as (x, z) . Now we are back to the case of maximal entanglement, as can be seen from the combined state of $\mathcal{Y}$ and $\mathcal{D}_Y(x)$ being $|z\rangle|z\rangle$.

⁸ This is because entanglement is a *basis-independent* feature of quantum states; in fact, it fulfills the definition of a *quantum resource*, i.e. something that can be spent to perform tasks <https://arxiv.org/abs/1806.06107>.

⁹ In fact, this is the observation that lead Hugh Everett to develop his many-worlds interpretation of quantum mechanics, since the randomness induced by wavefunction collapse can just as well be explained in terms of you entangling *yourself* with the quantum state you are observing. And why not? After all, you, too, are made of quantum stuff!

Of course, in between the two extremes of no information and maximal entanglement lies the world of *some* entanglement, as results from setting $\mathcal{Y}$ to some non-uniform superposition. Additionally, $\mathcal{Y}$ may itself be arbitrarily entangled with other subsystems belonging to the querying party, in which case it will be a mixed state from the point of view of the compressed oracle, and so their combined state will, after entangling, also be a mixed state.

Superpositions over input values. Finally, what about superpositions over x ? The easiest way to think about this is to adopt an Everettian worldview, and imagine the superposition state of $\mathcal{X}$ describing a superposition over several possible universes. Then, one can consider each universe separately: The branch of the wavefunction where $\mathcal{X}$ is in basis state $|x\rangle$ is the branch of the wavefunction where the querying party (who will in general be entangled with $\mathcal{X}$) decided to query the random oracle on the value x . And since these registers always remain in the primal domain, we don't have to grapple with the metaphysical implications of “changing the basis” on a collection of universes.

Thus, the result of such a query again depends on the state of $\mathcal{Y}$: If $\mathcal{Y}$ is in the uniform superposition, independently of $\mathcal{X}$, then it is in the uniform superposition in every universe, and nothing is ever stored and nothing is ever learned. If $\mathcal{Y}$ is set in a computational basis state, such as $|0^m\rangle$, then the result is a maximally entangled state, appearing as a classically uniform probability distribution over basis states in all universes. Finally, if $\mathcal{X}$ and $\mathcal{Y}$ are maximally entangled in some state $|x_1\rangle|y_1\rangle + |x_2\rangle|y_2\rangle + \dots$, for unique values of x_i and y_i , then inside each universe we are performing a purely classical query, and we are back to the situation that we first considered.

Repeated queries. Say the oracle started out in the empty state $|\mathcal{D}_\varepsilon\rangle$, and was then queried on the classical input state $|x\rangle|y\rangle$. Then we found that the result is the combined state

$$\sum_{\forall \hat{y} \in [2^m] \setminus \{0^m\}} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |x\rangle (H^{\otimes m} |\hat{y}\rangle) |\mathcal{D}_{x,\hat{y}}\rangle + \frac{1}{\sqrt{2^m}} |x\rangle (H^{\otimes m} |0^m\rangle) |\mathcal{D}_\varepsilon\rangle .$$

Now, what happens if the adversary queries the oracle on the registers $\mathcal{X}$ and $\mathcal{Y}$ a second time, without changing their in any way in the meantime? Well, first, the Hadamard transformation on $\mathcal{Y}$, being an involution, cancels out:

$$\sum_{\forall \hat{y} \in [2^m] \setminus \{0^m\}} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |x\rangle |\hat{y}\rangle |\mathcal{D}_{x,\hat{y}}\rangle + \frac{1}{\sqrt{2^m}} |x\rangle |0^m\rangle |\mathcal{D}_\varepsilon\rangle .$$

Next, for each basis state $|x\rangle|\hat{y}\rangle|\mathcal{D}_{x,\hat{y}}\rangle$ such that $\hat{y} \neq 0^m$, we will have that $D[x] \neq \perp$, and so S will do nothing, while for $\hat{y} = 0^m$ it will insert the element $(x, 0^m)$ into D . This gives us the state:

$$\sum_{\forall \hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |x\rangle |\hat{y}\rangle |\mathcal{D}_{x,\hat{y}}\rangle .$$

Then, E will have the following effect on the basis states:

$$|x\rangle|\hat{y}\rangle|(x, \hat{y}), \dots\rangle \xrightarrow{E} |x\rangle|y\rangle|(x, \hat{y} \oplus y), \dots\rangle = |x\rangle|y\rangle|(x, 0^m), \dots\rangle ,$$

and with the second application of S removing all entries with $\hat{y} = 0^m$, we will end up with the unentangled state

$$\sum_{\forall \hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |x\rangle |\hat{y}\rangle |\mathcal{D}_\varepsilon\rangle .$$

Transforming $\mathcal{Y}$ back to the primal domain then yields

$$|x\rangle|y\rangle|\mathcal{D}_\varepsilon\rangle ,$$

which is exactly where we started before the first query!

We see that the entanglement has been removed, and the value of x has been completely removed from the oracle's view. What's more, since this “forgetting” behaviour is shown to hold for any computational basis state, it must hold for *every* quantum state that is forwarded to the oracle twice in a row, making the compressed oracle *itself* an involution! Worse, not only does the oracle seem to “forget” that it was ever queried, but so does the querying party also “forget” the value of $H(x)$!

It may seem counter-intuitive, but this “forgetting” is in fact exactly what we need to happen (and was indeed a major roadblock towards lazy-sampling quantum random oracles prior to the development of this technique). To see why, consider the situation for “monolithic” random oracles, which are also involutions: Here, the classical query $|x\rangle|y\rangle$ returns the state $|x\rangle|y \oplus H(x)\rangle$. If the query is then repeated with the same registers without altering their state, the result will be $|x\rangle|y \oplus H(x) \oplus H(x)\rangle = |x\rangle|y\rangle$. The value of $H(x)$ has been forgotten!

“But”, I hear you say, “what if I store the value of $H(x)$ before I make the second query? Surely I won’t be forced to automatically ‘forget’ what the value was, simply because I query the oracle a second time!” And you’d be right—but notice that we are now talking about a different quantum state, namely the state $|x\rangle_{\mathcal{X}}|y \oplus H(x)\rangle_{\mathcal{Y}}|H(x)\rangle_{\mathcal{A}}$, where the $\mathcal{A}$ register is some storage register local to you. By storing $H(x)$ in this register (by first preparing it in the state $|y\rangle$ and then performing the m -bit CX gate from $\mathcal{Y}$ to $\mathcal{A}$), you *entangled* the two registers. And so, while the result of a second query to the monolithic oracle will indeed be the state $|x\rangle|y\rangle|H(x)\rangle$, the effect on the compressed oracle will be completely different to the “forgetting” behaviour outlined above. This can be seen from the fact that the Hadamard transformation, which previously operated on the pure, unentangled state $|y\rangle$, will now take the $\mathcal{Y}$ register to some other state than the one it was in during the first query, and the cancellation of the $\hat{y}$ values in the second query won’t happen.

How consistency is maintained. Now, let us return to the conundrum from the start of the section: Is it really true that forgetting x on one of 2^n points can never lead to problems? Well—yes and no. We’ll start with “yes” here, and leave the “no” for the next section.

Consider the following situation: An adversary $\mathbb{A}$ queries wants to know whether the oracle they have gotten access to is a monolithic oracle or a compressed oracle. They query it on the state $|x\rangle|y\rangle$, and then measure the y register, writing down the result. Then they query the oracle on the same state again. If the oracle is monolithic, then the second result is guaranteed to be the same as the first, and if the oracle is compressed, then it is easy to see that the result will be consistent as long as the measurement didn’t lead the database to collapse to the one out of 2^m branches where $\hat{y} = 0^m$. But what if it did?

For simplicity, let’s consider the case that $m = 1$, and calculate the exact state after each operation. In Sect. 4.2, we saw that the first three operations of the compressed oracle lead to the following state:

$$\sum_{\forall \hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |x\rangle |\hat{y}\rangle |D_{x,\hat{y}}\rangle ,$$

which for $m = 1$ we can write as

$$|x\rangle \frac{|0\rangle |D_{x,0}\rangle + (-1)^y |1\rangle |D_{x,1}\rangle}{\sqrt{2}} .$$

After applying S , we will have

$$|x\rangle \frac{|0\rangle |D_\varepsilon\rangle + (-1)^y |1\rangle |D_{x,1}\rangle}{\sqrt{2}} .$$

Next, we move the $\mathcal{Y}$ register back to the primal domain, which gives us the state

$$|x\rangle \frac{\frac{|0\rangle+|1\rangle}{\sqrt{2}} |D_\varepsilon\rangle + (-1)^y \frac{|0\rangle-|1\rangle}{\sqrt{2}} |D_{x,1}\rangle}{\sqrt{2}} = |x\rangle \frac{|0\rangle |D_\varepsilon\rangle + |1\rangle |D_\varepsilon\rangle + (-1)^y |0\rangle |D_{x,1}\rangle + (-1)^{1-y} |1\rangle |D_{x,1}\rangle}{2} ,$$

and send it back to $\mathbb{A}$. At this point, $\mathbb{A}$ measures the $\mathcal{Y}$ register. Let’s say they see the value 0, which they interpret as $y \oplus H(x) = 0$, and so conclude that $H(x) = y$. Now, to double check, they make the same query again.

From the point of view of the compressed oracle, the measurement collapsed the entangled state down to the following one, in which the $\mathcal{Y}$ register is guaranteed to be in the $|0\rangle$ state:

$$|x\rangle \frac{|0\rangle |D_\varepsilon\rangle + (-1)^y |0\rangle |D_{x,1}\rangle}{\sqrt{2}} = |x\rangle |0\rangle \frac{|D_\varepsilon\rangle + (-1)^y |D_{x,1}\rangle}{\sqrt{2}} .$$

But before returning the registers, to ensure an identical query, $\mathbb{A}$ reset the $\mathcal{Y}$ to $|y\rangle$, so that the state we receive is

$$|x\rangle |y\rangle \frac{|D_\varepsilon\rangle + (-1)^y |D_{x,1}\rangle}{\sqrt{2}} .$$

Now, the compressed oracle moves $\mathcal{Y}$ to the Hadamard domain:

$$|x\rangle \left(\frac{|0\rangle + (-1)^y |1\rangle}{\sqrt{2}} \right) \left(\frac{|\mathcal{D}_\varepsilon\rangle + (-1)^y |\mathcal{D}_{x,1}\rangle}{\sqrt{2}} \right) = |x\rangle \frac{|0\rangle |\mathcal{D}_\varepsilon\rangle + (-1)^y |1\rangle |\mathcal{D}_\varepsilon\rangle + (-1)^y |0\rangle |\mathcal{D}_{x,1}\rangle + |1\rangle |\mathcal{D}_{x,1}\rangle}{2},$$

applies S:

$$|x\rangle \frac{|0\rangle |\mathcal{D}_{x,0}\rangle + (-1)^y |1\rangle |\mathcal{D}_{x,0}\rangle + (-1)^y |0\rangle |\mathcal{D}_{x,1}\rangle + |1\rangle |\mathcal{D}_{x,1}\rangle}{2},$$

and then E:

$$\begin{aligned} & |x\rangle \frac{|0\rangle |\mathcal{D}_{x,0}\rangle + (-1)^y |1\rangle |\mathcal{D}_{x,1}\rangle + (-1)^y |0\rangle |\mathcal{D}_{x,1}\rangle + |1\rangle |\mathcal{D}_{x,0}\rangle}{2} \\ &= |x\rangle \frac{|0\rangle (|\mathcal{D}_{x,0}\rangle + (-1)^y |\mathcal{D}_{x,1}\rangle) + |1\rangle (|\mathcal{D}_{x,0}\rangle + (-1)^y |\mathcal{D}_{x,1}\rangle)}{2} \\ &= |x\rangle \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) \left(\frac{|\mathcal{D}_{x,0}\rangle + (-1)^y |\mathcal{D}_{x,1}\rangle}{\sqrt{2}} \right). \end{aligned}$$

Thereafter, S is applied a second time, before $\mathcal{Y}$ is moved back to the primal domain and shipped back to $\mathbb{A}$, leaving us with the state

$$|x\rangle |0\rangle \frac{|\mathcal{D}_\varepsilon\rangle + (-1)^y |\mathcal{D}_{x,1}\rangle}{\sqrt{2}}.$$

This is exactly the same state that we had after measuring! We see that the combined effect of putting $\mathcal{Y}$ back in the state $|y\rangle$ and repeating the query on the same point x actually acted like the identity on the state. In particular, this means that the query can be repeated indefinitely without any further effect on the state on the combined system whatsoever.¹⁰

Actually, since we showed that this works for both qubit states $|y\rangle$, and since all operations are performed bitwise anyway, this already implies that the oracle will exhibit the same behaviour for general superpositions, and for $m > 1$.

So, what was the crucial element that allowed consistency to be maintained? In my view, it is the fact that measuring $\mathcal{Y}$ only lead to a *partial* collapse of $\mathcal{D}$'s quantum state. In particular, some branches of the superposition still contained information about x , and (as so often happens in quantum theory) these branches just *happened* to be arranged in just the right state for phases to cancel out in such a way that consistency is maintained.

In other words, consistency crucially relied on the *database* remaining in a superposition, even after measurement. If we were to measure the *database*, then all bets would be off!

4.4 Extracting Queries

So naturally, that's exactly what we'll do next. After all, the usefulness of lazy sampling comes not only from the ability to efficiently simulate a random oracle, but also the ability to *see* what inputs the adversary is interested in!

First, if we want our simulation to remain perfect, we must avoid any and all measurements of the state of $\mathcal{D}$ as long as the game is running—only after it has ended can we perform our measurement, and see what we end up with.

So, does it work? *Are* we guaranteed that, if the adversary measures a series of queries and learns the values of $H(x_1), \dots, H(x_n)$, that measuring the database will give us the values $x_1, \dots, x_n$?

Well, *almost*—and this is where deleting all those x 's comes back to haunt us. There is namely a tiny probability that this leads to the adversary learning the point $H(x)$, which just happened to correspond to an x that we removed in at least some of the branches of the superposition, and that upon measuring the database, the branch we end up witnessing is exactly such a branch.

Thankfully, we can show that the probability of this unfortunate outcome is exponentially small in the length of the output.¹¹ This is what the next theorem, also proven by Zhandry [Zha19, Lemma 5], shows.

Theorem 2 (Extraction is imperfect). *Let the adversary $\mathbb{A}$ be a quantum algorithm with oracle access to a random oracle H , and that after at most q queries outputs $\ell \leq q$ tuples (x, y) with unique values for x , and let $\text{Adv}_{\mathcal{R}}(\mathbb{A}^H)$ be the probability that*

¹⁰ To check your understanding, you may want to verify that, had $\mathbb{A}$ kept $\mathcal{Y}$ in the $|0\rangle$ state in the second query, the result would be the output state $|y\rangle$, which is consistent with the measurement outcome $H(x) = y$.

¹¹ Which, provided we want our random oracles to be collision resistant, should be set large enough anyway.

1. For all tuples (x, y) , $H(x) = y$; and
2. The collection of tuples $\{(x, y)\}$ is contained in some set $\mathcal{S}$ that we care about.

Now, let the extractor $\mathbb{E}$ simulate the random oracle for $\mathbb{A}$ as a compressed random oracle. As stated in Thm. 1, the simulation will be perfect, and so we are guaranteed that with the same probability, $\mathbb{A}$ will output tuples as before.

After $\mathbb{A}$ halts, let $\mathbb{E}$ first move the output register of the oracle database to the primal domain by applying the Hadamard transformation to $\mathcal{D}_Y$, and then observe the database, obtaining a list of queries $\mathcal{D}$. Let $\text{Adv}_{\mathcal{S}}(\mathbb{E}^{\mathbb{A}})$ be the probability that

1. For all tuples (x, y) , $\mathcal{D}[x] = y \neq \perp$; and
2. The collection of tuples $\{(x, y)\}$ is contained in $\mathcal{S}$.

Then we have that, for all adversaries $\mathbb{A}$,

$$\sqrt{\text{Adv}_{\mathcal{S}}(\mathbb{A}^H)} \leq \sqrt{\text{Adv}_{\mathcal{S}}(\mathbb{E}^{\mathbb{A}})} + \sqrt{\frac{\ell}{2^m}},$$

which implies the slightly looser (but perhaps more practical) bound

$$\text{Adv}_{\mathcal{S}}(\mathbb{A}^H) \leq \text{Adv}_{\mathcal{S}}(\mathbb{E}^{\mathbb{A}}) + 2\sqrt{\frac{\ell}{2^m}} + \frac{\ell}{2^m}.$$

References

- AHU19. Andris Ambainis, Mike Hamburg, and Dominique Unruh. Quantum security proofs using semi-classical oracles. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 269–295. Springer, Cham, August 2019.
- BDG20. Mihir Bellare, Hannah Davis, and Felix Günther. Separate your domains: NIST PQC KEMs, oracle cloning and read-only indistinguishability. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 3–32. Springer, Cham, May 2020.
- BR93. Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93*, pages 62–73. ACM Press, November 1993.
- Deu85. David Deutsch. Quantum theory, the church-turing principle and the universal quantum computer. *Proc. R. Soc. Lond. A*, 400:97–117, 1985.
- GHHM21. Alex B. Grilo, Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Tight adaptive reprogramming in the QROM. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 637–667. Springer, Cham, December 2021.
- Jae24. Joseph Jaeger. Nonadaptive one-way to hiding implies adaptive quantum reprogramming. Cryptology ePrint Archive, Report 2024/797, 2024.
- NC10. Michael Nielsen and Isaac Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, second edition, 2010.
- PZ24. Jiaxin Pan and Runzhi Zeng. Selective opening security in the quantum random oracle model, revisited. In Qiang Tang and Vanessa Teague, editors, *PKC 2024, Part II*, volume 14603 of *LNCS*, pages 92–122. Springer, Cham, April 2024.
- Sho04. Victor Shoup. Sequences of games: a tool for taming complexity in security proofs. Cryptology ePrint Archive, Report 2004/332, 2004.
- Zha19. Mark Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 239–268. Springer, Cham, August 2019.

A A Brief Quantum Primer

Quantum theory arose as a branch of physics, and so much of the standard notational conventions are hand-wavy, and often inconsistently implemented across the literature. For the sake of clarity and self-containedness, we specify all our conventions below, providing a brief summary of the building blocks that make up quantum computing theory along the way; for more details, see the rightly celebrated introductory text by Nielsen and Chuang [NC10].

This chapter contains many footnotes: these are tangents with additional details that may interest the reader, but are inessential to the rest of the text. In particular, if a footnote made no sense to you, you should feel free to ignore it.¹²

The best way to get a feel for the logic of quantum computing, is to play around with it, and this chapter also provides several exercises towards this purpose, inserted at points in the text where you should be able to use them to test your understanding of the material presented so far. Solutions for some (but not all) of the exercises are provided in Sect. A.8.

A.1 Dirac Notation

In general, quantum states are complex-valued vectors in a Hilbert space (a vector space defined over functions), which may or may not be finite-dimensional.¹³ Quantum states are represented using the Dirac notation, in which a quantum state carrying the *label* ψ is written $|\psi\rangle$, its conjugate transpose is written $\langle\psi| := |\psi\rangle^\dagger := (|\psi\rangle^*)^T$, and its inner product with itself is written $\langle\psi|\psi\rangle := ||\psi||^2$. Quantum states are unit-norm, and so we have that $\langle\psi|\psi\rangle = 1$. For two not necessarily equal quantum states $|\psi\rangle$ and $|\varphi\rangle$, their inner product $\langle\psi|\varphi\rangle$ is a complex number α with norm $|\alpha|$ between zero and one, where “zero” means the states are orthogonal, and “one” means the states are identical (modulo an unobservable global phase factor $e^{i\theta}$).

This calculus is fine, but one may wonder, what *is* a quantum state—in the sense, what kind of mathematical object is it? The answer can be found in the statement that they live in *N-dimensional Hilbert space*: Hilbert spaces are vector spaces defined over *functions* in place of vectors, where the inner product between two functions f and g on the interval $[a, b]$ is defined to be $\int_{-\infty}^{\infty} f(x)^* g(x) dx$. This inner product defines a norm, and a notion of orthogonality, from which a vector space may be constructed. Quantum states, then, are the functions that arise as solutions to a differential equation describing the dynamics of the system, i.e. the Schrödinger equation in the non-relativistic, non-spin case; the Pauli equation in the case with spin (such as electrons), and the Dirac equation in the relativistic case (excluding gravity). They take some input variable corresponding to an observable, such as the position x , and returns the *amplitude* corresponding to that position; squaring this then gives the probability that observing the particle will yield x as a result.

If all of this sounds terribly complicated, then you understand why physicists grew so fond of Dirac’s notation, which neatly swept all of these horrible details under a rug of conceptual clarity!

Thankfully, things become a bit easier in the discrete case: the Hilbert space is now finite, and so it can be expressed in terms of a finite set of basis “vectors” (i.e., functions), and the inner product is now a sum rather than an integral. Further, functions now have finite domains, and may thus be written as N -length vectors instead, where each element corresponds to an input to the function, enabling us to employ our toolbox from linear algebra to study them (leveraging the isomorphism between $\mathcal{F}_{\{0,1\}^n \rightarrow \mathbb{C}}$ and $\mathbb{C}^{2^n}$). This is the approach taken in the quantum information sciences.

A.2 Qubit Basis States

Single-qubit states. We are interested in finite-dimensional quantum states, implemented on a collection of n qubits. A *qubit* is a quantum state existing in the two-dimensional Hilbert space characterized by the basis states $|0\rangle$ and $|1\rangle$. In physical implementations, these states may for example be mapped to the spin of an electron (“up”/“down”), or the two lowest energy states of an atom, or in general any

¹² Or ask me about it! I love explaining this stuff.

¹³ For example, consider the state $|p(x)\rangle$ describing the position $x \in [a, b]$ of particle p ; this will live in an infinite-dimensional Hilbert space characterized by the basis states $\{|p(x)\rangle\}_{x \in [a, b]}$.

two orthogonal quantum states, where two quantum states are orthogonal if and only if there exists a measurement¹⁴ procedure that perfectly distinguishes them.

It is worth emphasizing that the “0” and “1” here are *labels*, and do not say anything about the physical system in question other than that it contains (at least) two orthogonal quantum states, which are being identified with the two basis states $|0\rangle$ and $|1\rangle$.

Since two qubit basis states $|b_1\rangle$ and $|b_2\rangle$, for $b_1, b_2 \in \{0, 1\}$, the two states must be either identical (up to the aforementioned phase factor) or orthogonal, we have that

$$\langle b_1 | b_2 \rangle = \delta_{b_1 b_2} ,$$

where δ_{ij} is the Kronecker delta, equal to 1 if $i = j$, and 0 otherwise.

Multi-qubit states. In general, two quantum systems (i.e. states describing distinct physical systems) may be combined via the tensor product:

$$|\psi\rangle \otimes |\varphi\rangle := |\psi, \varphi\rangle .$$

If $|\psi\rangle$ describes an N -dimensional quantum system, and $|\varphi\rangle$ describes an M -dimensional quantum system, then the combined state $|\psi, \varphi\rangle$ describing the combined system will be NM -dimensional. Note how the dimensions are *multiplied* rather than added, as one might naïvely expect.

Applying this to qubits, given a collection of n qubits $|b_i\rangle$, each initialized to one of the two basis states (so that $b_i \in \{0, 1\}$), we may construct the combined state

$$\begin{aligned} |b_0\rangle \otimes |b_1\rangle \otimes \dots \otimes |b_{n-1}\rangle &:= |b_0\rangle |b_1\rangle \dots |b_{n-1}\rangle \\ &:= |b_0, b_1, \dots, b_{n-1}\rangle \\ &:= |b_0 b_1 \dots b_{n-1}\rangle . \end{aligned}$$

We will use any and all of the above conventions when denoting combined quantum states, choosing whichever one we deem to bring the most clarity and eligibility to the given context. (For example, non-binary labels will typically be separated by commas, while binary-valued labels will typically not be, in reference to string concatenation $b_0 b_1 := b_0 \| b_1$.)

Since each qubit is a two-dimensional quantum state, the combined state will have dimension $N := 2^n$. The resulting Hilbert space will be characterized by the computational¹⁵ basis states

$$\{|i\rangle\}_{i \in [N]} = \{|b_0 b_1 \dots b_{n-1}\rangle\}_{b_0, \dots, b_{n-1} \in \{0, 1\}} .$$

This exponentiality in the dimension of quantum states as the number of qubits increases sits at the core of quantum computers’ advantage over classical computers. As an illustrative example, the Hilbert space corresponding to a three-qubit quantum system is characterized by the eight computational basis states:

$$\begin{aligned} \{|i\rangle\}_{i \in [2^3]} &= \{|000\rangle, |001\rangle, |010\rangle, |011\rangle, |100\rangle, |101\rangle, |110\rangle, |111\rangle\} \\ &= \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \right\} . \end{aligned}$$

As in the single-qubit case, we see that we have orthonormality over the computational basis states:

$$\langle x | y \rangle = \delta_{xy} .$$

¹⁴ The words “observation” and “measurement” describe the same procedure. I prefer “observation” when speaking informally, as it captures the fact that you don’t need fancy lab equipment to collapse a quantum state—coming in contact with it in any way will suffice; but I will follow the standard nomenclature of “measurement” when talking about the technical formalism.

¹⁵ As with any vector space, Hilbert spaces permit an infinitude of basis choices; the “computational” basis is simply the name of the choice wherein basis states are labelled by classical bits (or more generally, bitstrings).

We also define notation for the repeated tensor product of identical states, so that e.g. initializing a set of n qubits to the $|0\rangle$ state may be written:

$$|0 \dots 0\rangle = \bigotimes_{i=0}^{n-1} |0\rangle := |0\rangle^{\otimes n}.$$

A.3 General Qubit States

Greek vs. Roman letters. We will use the letters $x, y, z, \dots$ to represent classical binary strings. Thus, a state $|x\rangle$ will always be understood to be a computational basis state. Whenever we are talking specifically about single-qubit computational basis states, we will use b to represent a single bit, and write $|b\rangle$.

General quantum states are traditionally labelled by letters taken from the middle of the greek alphabet, such as ψ, φ, ξ , etc. We will follow this tradition, or otherwise use a descriptive label relating to the specific context in which the state is being used. Quantum *amplitudes*, meanwhile, are written using letters taken from the start of the greek alphabet ($\alpha, \beta, \dots$).

Expanding a state in terms of the basis. A general single-qubit state may be written in terms of the computational basis states as follows:

$$|\psi\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle,$$

where the amplitudes α_i are complex numbers satisfying $|\alpha_0|^2 + |\alpha_1|^2 = 1$, (where the absolute value of a complex number α is defined as $|\alpha| = \sqrt{\alpha^* \alpha}$).

Whenever a qubit is in a state described by amplitudes α_0 and α_1 that are both non-zero, we say that the qubit is in a *superposition* over the values “0” and “1”. If we then *observe* the qubit, the probability that the observed outcome X is either 0 or 1 is then given by $\Pr[X = b] = |\alpha_b|^2 = |\langle\psi|b\rangle|^2$, and after observation, the superposition will have “collapsed” to the basis state corresponding to the observed value, $|b\rangle$.¹⁶

Likewise, a general, n -qubit quantum state may be written in terms of the computational basis states:

$$|\psi\rangle = \sum_{\forall i \in [2^n]} \alpha_i |i\rangle = \sum_{i=0}^{N-1} \alpha_i |i\rangle = \alpha_0 |0 \dots 00\rangle + \alpha_1 |0 \dots 01\rangle + \dots + \alpha_{N-1} |1 \dots 11\rangle,$$

where the *amplitudes* α_i again satisfy $\sum_{\forall i \in [2^n]} |\alpha_i|^2 = 1$.

When expanded over the basis states, the inner product of two quantum states $|\psi\rangle = \sum_{\forall i \in [N]} \alpha_i |i\rangle$ and $|\varphi\rangle = \sum_{\forall j \in [N]} \beta_j |j\rangle$ may be written

$$\langle\psi|\varphi\rangle = \sum_{\forall i, j \in [N]} \alpha_i^* \beta_j \langle i|j\rangle = \sum_{\forall i, j \in [N]} \alpha_i^* \beta_j \delta_{ij} = \sum_{\forall i \in [N]} \alpha_i^* \beta_i.$$

We see that we have recovered the inner product of complex-valued vectors, which is just the usual inner product of real-valued vectors with an extra complex conjugation.¹⁷ Thus we may apply the usual operational interpretation of the inner product, in which vectors are “perpendicular” when their inner product is 0, and “parallel” when it is 1. Together with the requirement that quantum states have unit

¹⁶ The discernible effect of observations on quantum states has led to a century of heated philosophical debate over how quantum theory should be interpreted: The term “collapse” used here is typically associated with the conservative “Copenhagen interpretation”, championed by Niels Bohr, and commonly characterized as taking a “shut-up-and-calculate” attitude towards the problem. Another popular interpretation is the many-worlds interpretation due to Hugh Everett, and favoured by father of quantum computing theory David Deutsch, among others. Instead of the quantum state “collapsing” to become “classical”, this interpretation would say that both terms are ill-defined, and that the universe should instead be understood to be nothing but one big superposition state over all worlds that might have arisen from the initial conditions of the Big Bang; and that what actually happened when you made your observation, was that the quantum state describing the subsystem you were studying and the quantum state describing the subsystem containing *you* became correlated via the process known as *entanglement*. (In case you’re wondering, I place my own bet on the *relational* interpretation due to Carlo Rovelli, which may be described as “many-worlds without the multiverse”.)

¹⁷ This is because inner products on any vector space must satisfy conjugate symmetry, $\langle x, y \rangle = \langle y, x \rangle^*$; see https://en.wikipedia.org/wiki/Inner_product_space.

norm, this means that an inner product of 1 implies that the two quantum states are actually identical, (up to the aforementioned unobservable phase factor).

When expanding quantum states in terms of basis states in the manner above, we will sometimes implicitly exclude terms for which $\alpha_i = 0$, writing instead

$$|\psi\rangle = \sum_i \alpha_i |x_i\rangle := \sum_{\forall i \in \mathcal{I}} \alpha_i |x_i\rangle ,$$

where $\mathcal{I} = [\ell]$ (for some $\ell \in \mathbb{N}$) indexes the set $\{|x\rangle \mid x \in [N] \wedge \langle \psi | x \rangle \neq 0\}$.

A.4 Qubit Registers

Given a collection of n qubits, we will find it useful to divide it into *registers* of specified sizes, for example to separate distinct elements in an input. We may use subscripts to refer to specific registers, unless the register we are referring to is clear from context.

As an example, let's say we have n qubits, all initialized to $|0\rangle$, and that we want to put the i 'th qubit in the state $|\psi\rangle$ the j 'th qubit in the state $|\varphi\rangle$. In other words, we want to do the following:

$$|0 \dots 0\rangle \longrightarrow |0 \dots 0, \psi, 0 \dots 0, \varphi, 0 \dots 0\rangle ,$$

or, more precisely,

$$|0\rangle^{\otimes n} = |0\rangle^{\otimes i} |0\rangle |0\rangle^{\otimes(j-i)} |0\rangle |0\rangle^{\otimes(n-j)} \longrightarrow |0\rangle^{\otimes i} |\psi\rangle |0\rangle^{\otimes(j-i)} |\varphi\rangle |0\rangle^{\otimes(n-j)} .$$

By indexing the qubit states, this may be written more succinctly:

$$|0\rangle_i |0\rangle_j \longrightarrow |\psi\rangle_i |\varphi\rangle_j ,$$

where $i, j \in [n]$, and by convention the state of any qubit not listed is unchanged by the operation.¹⁸

A.5 Unitary Operations

Since we are working over finite-dimensional Hilbert spaces, quantum operations, that is, evolutions of quantum states, may be represented as $N \times N$ -dimensional matrices. What properties should they have?

Clearly, any operation that we perform on a quantum state (*except* observing them—we'll get to that!) should yield another valid quantum state as a result. Since quantum states must be unit-norm, this means that any operation we perform on them must preserve the norm. This fact turns out to be sufficient to restrict the space of allowed operations to the unitary matrices, i.e. invertible square matrices U satisfying $U^{-1} = U^\dagger$.

Exercise 1. Prove that requiring operations to be norm-preserving is equivalent to restricting the set of legal operations to the set of unitary matrices. (Solution is given in Sect. A.8.)

We will use (sans-serif) capital roman letters to refer to quantum operations, in reference to their matrix representation (see Table 1 for a summary of our notational conventions). Whenever the qubit(s) that the operation is being applied to is not clear from context, we provide the indices of the qubits in the subscript, so for example U_i for a single-qubit operation applied to qubit i , and $U_{i,j}$ for a two-qubit operation applied to qubit i and j . (Note that in general, $U_{i,j} \neq U_{j,i}$.)

It is often useful to refer to operations in terms of the effect they have on the starting state, and so we define $|\psi\rangle \xrightarrow{U} |\varphi\rangle$ to mean that we perform the operation $|\psi\rangle \longrightarrow U|\psi\rangle := |\varphi\rangle$.

Some elementary unitary transformations, known as quantum *gates*, include

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} , \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} , \quad H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} , \quad CX = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} ,$$

where the matrices shown are the computational-basis representations of the general unitary transformations.

¹⁸ In particular, this means that this operation would not be well-defined if qubit i and/or j were entangled with any other qubits; we will remedy this situation in Sect. A.6.

Exercise 2. Write down “truth tables” for each of the above gates by writing down their effect on the (single-qubit resp. two-qubit) computational basis states. (E.g., $|0\rangle \xrightarrow{X} |1\rangle$, etc.)

The “C” in CX is short for “controlled”, where the value of the first qubit *controls* the operation on the second qubit: $|1b\rangle \xrightarrow{CX} |1(1-b)\rangle$, but $|0b\rangle \xrightarrow{CX} |0b\rangle$. Any single-qubit operation U defines a two-qubit controlled version of the same operation in the manner shown, which we will refer to as CU.

Exercise 3. Starting with the two-qubit state $|\psi\rangle_{0,1} = |0\rangle_0 |0\rangle_1$, apply the operations H_0 and $CX_{0,1}$. What are the possible outcomes of observing the resulting state, and what are their probabilities? Compare with the result of applying the single-qubit gates H_0 and H_1 instead.

Exercise 4. Write a new truth table for X, Z, and H, only now in terms of the alternative “Hadamard” basis

$$\left\{ |+\rangle := \frac{|0\rangle + |1\rangle}{\sqrt{2}}, |-\rangle := \frac{|0\rangle - |1\rangle}{\sqrt{2}} \right\}$$

instead. Compare with the truth table from Exercise 2. What has changed?

Exercise 5. Give the (computational-basis) matrix representation of a unitary operation that acts on the Hadamard basis from Exercise 4 in the same way that CX acts on the computational basis. What would we call this unitary, following the conventions introduced so far?

The SWAP gate. Another two-qubit unitary gate that will turn out to be useful is the SWAP gate, defined as $|01\rangle \xrightarrow{\text{SWAP}} |10\rangle$, $|10\rangle \xrightarrow{\text{SWAP}} |01\rangle$, and acting like the identity on $|00\rangle$ and $|11\rangle$. Applying this to general qubit states, we get

$$\begin{aligned} (\alpha_0 |0\rangle + \alpha_1 |1\rangle) \otimes (\beta_0 |0\rangle + \beta_1 |1\rangle) &= \alpha_0 \beta_0 |00\rangle + \alpha_0 \beta_1 |01\rangle + \alpha_1 \beta_0 |10\rangle + \alpha_1 \beta_1 |11\rangle \\ \xrightarrow{\text{SWAP}} \alpha_0 \beta_0 |00\rangle + \alpha_0 \beta_1 |10\rangle + \alpha_1 \beta_0 |01\rangle + \alpha_1 \beta_1 |11\rangle &= (\beta_0 |0\rangle + \beta_1 |1\rangle) \otimes (\alpha_0 |0\rangle + \alpha_1 |1\rangle). \end{aligned}$$

Thus, we see that the SWAP gate has the effect of swapping the states of two qubits with each other.¹⁹

Exercise 6. Give the (computational-basis) matrix representation of the SWAP gate.

Transformations in space vs. time. You may have noticed that we are working with two different kinds of product when dealing with unitary matrices: The matrix product $U_1 U_2$, and the tensor product $U_1 \otimes U_2$. When should each kind of product be used?

To illustrate, let $|\psi\rangle$ and $|\varphi\rangle$ be two single-qubit states, combined in the two-qubit state $|\psi, \varphi\rangle := |\psi\rangle \otimes |\varphi\rangle$. Then, if U_1 and U_2 are single-qubit transformations (i.e. 2×2 unitary matrices), and you want to apply U_1 to $|\psi\rangle$ and U_2 to $|\varphi\rangle$, you might write

$$(U_1 \otimes U_2)(|\psi\rangle \otimes |\varphi\rangle) := U_{1,2} |\psi, \varphi\rangle.$$

The matrix $U_{1,2} := U_1 \otimes U_2$ will be a two-qubit transformation (i.e. a 4×4 unitary) that combine the effect of the two single-qubit transformations to apply them in parallel (that is, in the same time step).

Next, imagine instead that you have two two-qubit unitaries (4×4 matrices) U_1 and U_2 , and you want to apply U_1 first, and then U_2 . In other words, you want something to the effect of

¹⁹ We deliberately avoided specifying registers in this example, as the question of whether it was the quantum *states* that were swapped, or the physical qubits themselves that switched places in the register, is a meta-physical question without relevance in this model. This is because by definition, qubits must be physically indistinguishable quantum systems. If they weren’t, this distinguishability could be used to construct a larger basis, say, $\{|0_a\rangle, |1_a\rangle, |0_b\rangle, |1_b\rangle\}$. If qubit a could only be in the a basis states, and qubit b could only be in the b basis states, then this would deny any quantum-mechanical interference between the amplitudes of qubit a and those of qubit b .

Similar observations remain true in real-world systems, although there the continuity of positions and momenta in space (with their corresponding infinite-dimensional Hilbert spaces) usually means that one *can* tell whether the particles from which the qubits are constructed were physically moved (as for trapped-ion or neutral atom systems), or whether the qubits were made to interact via a particular mechanism (as for superconducting qubit systems).

$$|\psi(0), \varphi(0)\rangle \xrightarrow{U_1} |\psi(1), \varphi(1)\rangle \xrightarrow{U_2} |\psi(2), \varphi(2)\rangle,$$

where $|\psi(t)\rangle$ denotes the evolved state at time step $t \in \mathbb{N}$, and $|\psi(0)\rangle := |\psi\rangle$. This is then given by

$$U_2 U_1 |\psi(0), \varphi(0)\rangle = U_2 (U_1 |\psi(0), \varphi(0)\rangle) := U_2 |\psi(1), \varphi(1)\rangle := |\psi(2), \varphi(2)\rangle.$$

Note the reversed order of operations!

As before, we could define a unitary $U_{1,2} := U_1 U_2$. Thus, even though we want to say that the application of U_1 and U_2 happens in two time steps, we see that the time of application can be “collapsed” to a single time-step! This means that when counting time steps in the analysis of quantum algorithms, we first have to fix some Turing-universal set of elementary quantum gates that are all said to happen take one time-step to apply, and require that all other unitaries are constructed from this gate set, in series and/or in parallel.

To summarize, the tensor product is used when applying unitaries in *parallel*, while the matrix product is used when applying unitaries in *series*.

Exercise 7. Construct the SWAP gate as a series of CX gates.

A.6 Quantum Entities, Entanglement, and Towards a Quantum Pseudocode

So far, we have defined notation for computational basis states $|x\rangle$, general quantum states $|\psi\rangle$, unitary transformations U , and the the result of applying a unitary transformation U on a quantum state $|\psi\rangle$, written $|\psi\rangle \xrightarrow{U} |\varphi\rangle$; see Table 1 for an overview.

And still this isn’t enough. This is because we are going to be talking about communicating parties. Consider the situation that Alice has two quantum bits initialized to the zero state, so that her state is $|\psi\rangle = |0\rangle_0 |0\rangle_1$. Now she wants to send her second qubit register, $|0\rangle_1$, over to Bob, who is going to apply some single-qubit unitary to it, and then send it back. We might write this as $|0\rangle_1 \xrightarrow{U_1} |\xi\rangle_1$, where $|\xi\rangle_1$ is some unknown, single-qubit state, and say that Bob returns $|\xi\rangle_1$ to Alice at the end of the protocol.

But what if Alice had prepared her two qubits in the state $|\varphi\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$ instead? Then, the two-qubit state is *non-separable*, meaning that it can no longer be cleanly factored into a tensor product of two single-qubit states. In the world of quantum physics, this means that the two qubits are *entangled*—a fancy word describing a phenomenon with many deep consequences for physics and philosophy alike—but mathematically speaking it is nothing but the quantum generalization of correlated probabilities.²⁰ Alice can still send her second qubit to Bob, and Bob can still apply the operation U to it; our problem is just that with the tools presented so far, we have no way²¹ of writing the operation that Bob is performing on $|0\rangle_1$ without referencing the entire state, like so:

$$\frac{|00\rangle_{0,1} + |11\rangle_{0,1}}{\sqrt{2}} \longrightarrow (\mathbb{1}_0 \otimes U_1) \left(\frac{|00\rangle_{0,1} + |11\rangle_{0,1}}{\sqrt{2}} \right).$$

What we want, is to be able to write the operations of Alice and Bob without having to reference qubits that they don’t have access to, regardless of whether or not their quantum states are entangled. In other words, what we need is a better *quantum pseudocode*.

²⁰ In other words, the term “separable” here carries exactly the same meaning as the term “product” does when talking about classical probability distributions. More precisely, if an arbitrary quantum state is *separable* along some dividing line, such that $|\psi\rangle = |\varphi\rangle \otimes |\xi\rangle$, then the probability distribution over the possible outcomes of observing $|\psi\rangle$ will form a product distribution, factorizable along the same dividing line. This remains true regardless of which basis you use to observe the state, implying that entanglement is an observation-independent (and therefore “objective”) property of the state.

²¹ For the readers ahead of the curve: Yes, one may reference one half of an entangled state using the generalized notion of a *mixed state*, by going to the density matrix formalization of quantum mechanics, which generalizes quantum states by combining them with classical probabilities. (Informally speaking, a density matrix is nothing but a probability distribution over quantum states.) The reason we avoid this solution is that mixed states are not *unique*, in the sense that there are many (larger) pure states that might have given rise to a particular density matrix, and so there would be many global states that could have led to Bob receiving a particular mixed state. Therefore, density matrices lack the expressive power to specify how Bob’s operations on the qubits available to him may, through entanglement, affect the state of Alice’s qubits.

Qubits and registers. Let q be a physical qubit, in the sense of being some kind of particle or quantum system existing in the physical world. Now, guided by our intuition, we want to say that Alice, instead of sending a quantum *state* to Bob, actually sends him the *physical qubit* q .²² Anyone who receives a qubit, should also gain the ability to apply any transformation or measurement on it, while those who sent it should lose the ability to do so, unless and until the qubit is returned to them. Crucially, we should be able to describe these actions *locally*, i.e. without reference to any qubits *not* in the possession of the receiving party.

So, with that goal in mind, let us introduce a little more formalism.

Definition 6 (Qubit). A qubit q is a two-dimensional physical system, i.e. a physical system associated with an observable $\hat{q}$, taking values in $\{0, 1\}$. Each qubit is additionally associated with a unique register address $i \in \mathbb{N}$.²³

Let us refer to the qubit sitting at address i as q_i . If qubit q_i is in not entangled with any other qubit (i.e. is in a pure state), then its state may be denoted $|q_i\rangle$ (equivalent to $|q_i\rangle_i$). If $|q_i\rangle = |\psi\rangle$ for some particular single-qubit state $|\psi\rangle$, then this may be denoted $|\psi\rangle_i$. Likewise, the state of qubit i at time t may be denoted $|q_i(t)\rangle$. For the sake of illustration, let's say that at time t the unitary U was applied to q_i ; this would then be written $|q_i(t)\rangle \xrightarrow{U_i} |q_i(t+1)\rangle$. Likewise, if q was initialized to $|0\rangle$, then $|q(0)\rangle = |0\rangle$.

Next, let us define our “universe”, that is, the full, undivided quantum memory which will act as a stage for all our plays.

Definition 7 (Quantum memory). An n -qubit quantum memory is an n -element list of qubits $\mathcal{Q} = (q_1, \dots, q_n)$ comprising a quantum system that is closed in the following sense: There are no operations that may be performed on the qubits q_i such that $\mathcal{Q}$ can no longer be said to be in a pure state.

In particular, $\mathcal{Q}$ can not get entangled with any system outside of itself—though it may be *observed*, after which it will immediately collapse to the observed basis state.²⁴

The requirement that $\mathcal{Q}$ is always in a pure state also implies that state preparation must be *deterministic* (or at least posterior to any randomness involved in the procedure): Flipping a (classical) coin and saying that you prepare a qubit in the state $|1\rangle$ on heads and $|0\rangle$ on tails would, when conditioning on the result of the coin flip, put $\mathcal{Q}$ in a mixed state, and this must therefore be disallowed. Thankfully, due to Turing-universality, we can make this restriction without loss of generality.

Thus, our N -dimensional “global” quantum state $|\mathcal{Q}\rangle$ will always be well-defined, giving us a “smallest” element q and a “largest” element $\mathcal{Q}$. That means it is time to define our in-betweens.

Definition 8 (Quantum register). An m -qubit quantum register $\mathcal{R}$ is a subsystem of the n -qubit quantum memory $\mathcal{Q}$, that is, an ordered sub-list of $\mathcal{Q}$, where by ordered we mean that if $\mathcal{R} = (q_{i_0}, q_{i_1}, \dots, q_{i_{m-1}})$ then $i_0 < i_1 < \dots < i_{m-1}$.

If the qubits in $\mathcal{R}$ are not entangled with any other subsystems of $\mathcal{Q}$, then the system $\mathcal{R}$ is in a pure state, and we may write $|\mathcal{R}\rangle$. In a slight overload of notation, let $[\mathcal{R}]$ denote the set $\{i_j\}_{j \in [m]}$ of qubit addresses associated with the qubits in $\mathcal{R}$; then $|\mathcal{R}\rangle$ is given by

$$|\mathcal{R}\rangle := \bigotimes_{\forall i \in [m]} |\mathcal{A}[i]\rangle := \bigotimes_{\forall i \in [\mathcal{R}]} |\mathcal{Q}[i]\rangle = |q_{i_0}, \dots, q_{i_{m-1}}\rangle.$$

If $|\mathcal{R}\rangle = |\psi\rangle$ for some specific state $|\psi\rangle$, this may be denoted $|\psi\rangle_{[\mathcal{R}]}$, which, having satisfied ourselves that it is well-defined, we will immediately shorthand to $|\psi\rangle_{\mathcal{R}}$.

²² In real-world quantum technologies, qubits are typically stored in one kind of system (e.g. neutral atoms) and sent using another kind (usually photons), but due to the non-cloneability of quantum states, together with the universal ability to transfer any quantum state from one qubit to another via the SWAP gate, this simplified model is without loss of generality.

²³ In the formalism of quantum Turing machines given by Deutsch [Deu85], the addresses correspond to values of the *address observable* $\hat{x}$ (except that we will limit ourselves to finite quantum memory by letting the set of possible addresses be $[n]$ rather than $\mathbb{Z}$).

²⁴ Of course, from the Everettian point of view, observing a state is to entangle yourself with it.

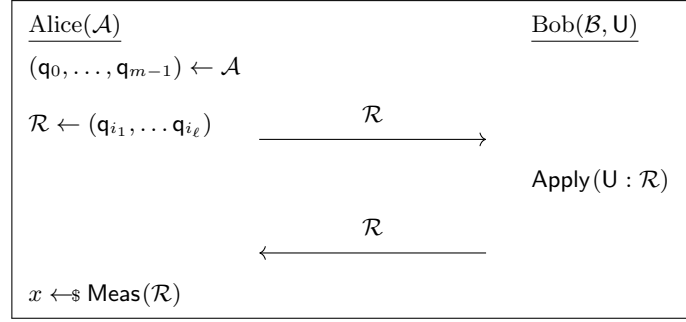


Fig. 3. Alice sends register $\mathcal{R}$ to Bob, who applies unitary U to it and sends it back to Alice, who finally measures it and returns the observed value. The registers $\mathcal{A}$ and $\mathcal{B}$ together make up the global quantum system $\mathcal{Q}$, left implicit in this figure.

Applying operations. But of course, the starting point of this entire exercise was that we *cannot* write the state of a register in terms of its own constituents in the general, entangled case, and so we still need an alternative way to denote operating on it.

Definition 9 (Operating on registers). Let $\mathcal{Q}$ be an n -qubit quantum memory in quantum state $|\mathcal{Q}\rangle$, let the register $\mathcal{R}$ be an m -qubit subsystem of $\mathcal{Q}$, and let U be an $M \times M$ unitary matrix.

Next, define $U_{\mathcal{R}}$ to be the $N \times N$ unitary matrix that has the effect of applying U on the qubits in $\mathcal{R}$ and the identity on the qubits in $\mathcal{Q} \setminus \mathcal{R}$. Specifically:

$$U_{\mathcal{R}} := \text{SORT}^\dagger([\mathcal{R}], [\mathcal{Q} \setminus \mathcal{R}]) (U \otimes \mathbb{1}) \text{SORT}([\mathcal{R}], [\mathcal{Q} \setminus \mathcal{R}]),$$

where $\text{SORT}([\mathcal{R}], [\mathcal{Q} \setminus \mathcal{R}])$ is the $N \times N$ unitary matrix, composed of a series of SWAP gates, that reorders the qubits in $\mathcal{Q}$ such that the qubits of $\mathcal{R}$ precede the qubits of $\mathcal{Q} \setminus \mathcal{R}$; $\text{SORT}^\dagger$ reverses the operation of SORT ; and $\mathbb{1}$ is the $2^{n-m} \times 2^{n-m}$ identity matrix.

Then, the application of U to the qubits in register $\mathcal{R}$ is captured in the method $\text{Apply}_{\mathcal{Q}}(U : \mathcal{R})$, defined as

$$\text{Apply}_{\mathcal{Q}}(U : \mathcal{R}) : |\mathcal{Q}\rangle \longrightarrow U_{\mathcal{R}} |\mathcal{Q}\rangle.$$

Whenever $\mathcal{Q}$ is clear from context, we will drop the subscript and simply write $\text{Apply}(U : \mathcal{R})$.

With this definition, it is clear that Alice *may* only be operating on the qubits that are available to her, and yet those very same operations may be affecting distant parts of $\mathcal{Q}$ through entanglement, which is now captured in $\text{Apply}_{\mathcal{Q}}(U : \mathcal{R})$ being a well-defined evolution of the global pure state $|\mathcal{Q}\rangle$.

Interacting entities. With these pieces in place, all that's missing is to define what we mean when we say that “Alice sends a qubit register to Bob”. We will do this by letting Alice and Bob each have access to the registers $\mathcal{A}$ and $\mathcal{B}$, respectively, which will themselves be non-overlapping subregisters of the total system $\mathcal{Q}$.

Definition 10 (Transferring a subregister). Let $\mathcal{A}$ and $\mathcal{B}$ be non-overlapping qubit registers. A subregister $\mathcal{R}$ is said to be transferred from $\mathcal{A}$ to $\mathcal{B}$, denoted $\mathcal{A} \xrightarrow{\mathcal{R}} \mathcal{B}$, if the elements of $\mathcal{R}$ are simultaneously removed from $\mathcal{A}$ and added to $\mathcal{B}$:

$$\mathcal{A} \xrightarrow{\mathcal{R}} \mathcal{B} \iff \mathcal{A} \leftarrow \mathcal{A} \setminus \mathcal{R}, \quad \mathcal{B} \leftarrow \mathcal{B} \cup \mathcal{R}.$$

Then, Alice sending register $\mathcal{R}$ to Bob simply amounts to transferring $\mathcal{R}$ from $\mathcal{A}$ to $\mathcal{B}$.

Finally, we have reached our goal: A quantum pseudocode rich enough to draw up well-defined interaction diagrams, like the one shown in Fig. 3.

A.7 Formalizing Measurements

Well—not quite. There is still one piece of formalism that needs to be dealt with before we can call it a day: Observations. Our goal is to make the $\text{Meas}(\mathcal{R})$ method in Fig. 3 well defined, in much the same manner as we did with $\text{Apply}(U : \mathcal{R})$ —but first we will need to take a step back: For, if observations are not unitary operations, what *are* they?

Projectors. The answer may be gleaned from the fact, stated earlier, that *upon observing a state, the quantum state collapses to the observed state*. This means that if you observe the state twice in a row (using the same measurement basis, to be defined shortly), then the outcome will be the same. The state has already collapsed; it can't collapse *more*.

If we call such an operation P , then we can write this insight as

$$P|\psi\rangle = P(P|\psi\rangle) = P^2|\psi\rangle \iff P^2 = P.$$

Such operators are known as *projectors*, so named because they have the effect of *projecting* a higher dimensional object onto a lower-dimensional hyperplane—or, in the case of global observations (i.e. measuring every qubit at once), all the way down to one of the basis states.

For single-qubit states, it is easy to verify that the following operators satisfy this definition:

$$P_0 := |0\rangle\langle 0| = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad P_1 := |1\rangle\langle 1| = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

From the matrix representations above, we see that effect of applying P_0 to a single-qubit state is to *single out* the $|0\rangle$ state from the superposition:

$$P_0|\psi\rangle = P_0(\alpha|0\rangle + \beta|1\rangle) = \alpha|0\rangle,$$

and applying P_0 on it again thereafter will have no effect, while applying P_1 will simply yield the zero vector. What's more, we see that $P_0 + P_1 = \mathbb{I}_2$. Clearly, these projectors will be *orthogonal* in the sense that they satisfy $P_i P_j = \delta_{ij} P_i$.

On the other hand, these matrices are manifestly *not* unitary, as seen from the fact that they are not even invertible. This reflects the irreversible loss of information induced by observation: After measuring a state, there is no way to recover the full pre-collapse state, other than to re-create it from scratch!

Let us next define the projector $P_{\mathcal{S}}$ onto some subspace $\mathcal{S}$ of the full N -dimensional Hilbert space, as specified by a subset of the computational basis states $\{|x_i\rangle\}_{\forall i \in [k]}$ (where $k \leq N$). This will simply be the sum of the relevant basis projectors:

$$P_{\mathcal{S}} = \sum_{\forall i \in [k]} |x_i\rangle\langle x_i| = \sum_{\forall i \in [k]} P_{x_i}.$$

The larger the sum, the “gentler” the measurement.

Example 1. Let $|\psi\rangle = \sum_{i \in [8]} \alpha_i |i\rangle$ be a three-qubit state. Then the projector P_{00b} , given by

$$P_{00b} = |000\rangle\langle 000| + |001\rangle\langle 001|$$

“selects” the states in the superposition in which the first and second qubits are zero:

$$P_{00b}|\psi\rangle = \alpha_0|000\rangle + \alpha_1|001\rangle.$$

We see that $|\psi\rangle$ has been *projected* from 8-dimensional Hilbert space down to the 2-dimensional subspace spanned by $|000\rangle$ and $|001\rangle$.

Unlike the basis-state projectors P_i , two general projectors $P_{\mathcal{S}}$, $P_{\mathcal{T}}$ need *not* be orthogonal, as subspaces $\mathcal{S}$ and $\mathcal{T}$ may be spanned by an overlapping set of basis vectors.

Exercise 8. Let P_{0b0} be given by

$$P_{0b0} = |000\rangle\langle 000| + |010\rangle\langle 010|,$$

and show that P_{0b0} and P_{00b} are non-orthogonal, where P_{00b} is as given in Example 1.

We will for the most part be interested in sets of orthogonal projectors. From the perspective of the computational basis states $|x\rangle$, this just means that no two projectors “accept” the same string x as part of their respective selections. For measurements, this translates to no measurement outcome being classified as “undecided”.

Projectors may not be unitary, but it is easy to see that they are *Hermitian*, meaning that they satisfy $P^\dagger = P$. For the computational basis projectors, this follows trivially from the matrices being real

and symmetric, but it also holds for general choices of basis states: Let P_φ be the projector that projects a state $|\psi\rangle$ onto the one-dimensional subspace spanned by the state $|\varphi\rangle$. Then,

$$P_\varphi^\dagger = (|\varphi\rangle\langle\varphi|)^\dagger = |\varphi\rangle\langle\varphi| = P_\varphi.$$

Finally, a set of projectors $\{P_{\mathcal{S}_i}\}_{\forall i \in [k]}$ is said to be *complete* if $\sum_{\forall i \in [k]} P_{\mathcal{S}_i} = \mathbb{1}$. In particular, the set of computational-basis projectors $\{P_i\}_{i \in [N]} = \{|i\rangle\langle i|\}_{i \in [N]}$ is easily seen to be complete: Each element in the sum will simply add one 1 to a previously-zero spot on the diagonal, and zero everywhere else; and the sum has N elements, so the full diagonal is covered. In the context of measurements, completeness reflects that outcome probabilities should sum to one.

Exercise 9. Provide a set of projectors such that, when taken together with P_{00b} from Example 1, the set is orthogonal and complete.

Projective measurements. However, $P|\psi\rangle$ will not itself be a quantum state, since it will not have unit norm (unless $P = \mathbb{1}$, that is). Additionally, we have not yet touched on the concept of measurement *outcomes*. What are the values that are actually observed? Something more is needed.

Let's say that the outcome of some measurement is m_x , which we interpret as the string x . We can formalize this by letting M be a *measurement operator* such that

$$|\psi\rangle \xrightarrow{M} m_x |x\rangle,$$

for one of several possible strings x , via some as-yet-undefined process. We can also make it more general, by checking which of several subspaces $|\psi\rangle$ belongs to

$$|\psi\rangle \xrightarrow{M} m_{\mathcal{S}_i} |\psi_{\mathcal{S}_i}\rangle,$$

where $|\psi_{\mathcal{S}_i}\rangle$ is the state that results from projecting $|\psi\rangle$ onto subspace $\mathcal{S}_i$.

Note that this measurement operator is *not* a projector, since repeated applications of M will give back increasing powers of m_x . However, provided that the measurement outcomes are all *orthogonal* (again meaning that none of the possible outcomes would be classified as “undecided”), we are guaranteed that there exists a complete set of orthogonal projectors $\{P_{\mathcal{S}_i}\}_{\forall i \in [k]}$ such that

$$M = \sum_{\forall i \in [k]} m_{\mathcal{S}_i} P_{\mathcal{S}_i}.$$

In linear algebra terms, this defines the *spectral decomposition* of the operator M , with the eigenvalues $m_{\mathcal{S}_i}$.

In the case that you are just reading off the values of n qubits directly in what we'll refer to as a “computational-basis measurement”, this will be

$$M = \sum_{\forall i \in [N]} m_i P_i = \sum_{\forall i \in [N]} m_i |i\rangle\langle i|,$$

which, when expressed in the computational basis, is just a sum of N matrices with the single non-zero element m_i in the i 'th position along its diagonal.

Finally, then, we are ready to formally define our measurement procedure.

Definition 11 (Projective measurement). Let i be an n -length bitstring, let $\{P_{\mathcal{S}_i}\}_{\forall i \in [k]}$ be a complete, orthogonal set of projectors (projecting onto the orthogonal subspaces $\mathcal{S}_i$ of the full Hilbert space), and let $M := \sum_{\forall i \in [k]} m_{\mathcal{S}_i} P_{\mathcal{S}_i}$ be a projective measurement operator. Then, measuring a state $|\psi\rangle$ under M yields outcome $\mathcal{S}_i$ with probability:

$$\Pr[|M|\psi\rangle = m_{\mathcal{S}_i}] = |P_{\mathcal{S}_i}|\psi\rangle|^2,$$

and, conditioned on $\mathcal{S}_i$ being observed, the act of measuring transforms the state as follows:

$$|\psi\rangle \xrightarrow{M} \frac{P_{\mathcal{S}_i}|\psi\rangle}{|P_{\mathcal{S}_i}|\psi\rangle|}.$$

The value $P_{\mathcal{S}_i}|\psi\rangle := \alpha_{\mathcal{S}_i} \in \mathbb{C}$ is known as the amplitude of observing outcome $\mathcal{S}_i$.

Again specializing to the case of measuring n qubits in the computational basis and reading off the resulting string, we see that the probability of observing any particular string x is given by

$$\Pr[|M|\psi\rangle = m_x] = |\mathbf{P}_x |\psi\rangle|^2 = |\alpha_x |x\rangle|^2 = |\alpha_x|^2,$$

with the post-measurement state

$$|\psi\rangle \xrightarrow{M} \frac{\mathbf{P}_x |\psi\rangle}{|\mathbf{P}_x |\psi\rangle|} = \frac{\alpha_x |x\rangle}{|\alpha_x |x\rangle|} = \frac{\alpha_x}{|\alpha_x|} |x\rangle \longrightarrow |x\rangle,$$

as required (where the final step does away with the unobservable global phase $\frac{\alpha_x}{|\alpha_x|} = e^{i\theta}$).

You may wonder, what *are* the values of the measurement outcomes, m_x ? Let us look at single-bit measurements: m_0 will signal outcome $|0\rangle$, and m_1 will signal outcome $|1\rangle$. Though tempting, we surely can't set $m_b = b$, since the outcome $|0\rangle$ would then multiply away the entire state! Instead, the canonical choice²⁵ is to let $m_0 = 1$ and $m_1 = -1$. Then,

$$M = m_0 P_0 + m_1 P_1 = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

You may recognize this matrix from earlier: It's the Z gate! In fact, computational-basis measurements are often referred to as “ Z measurements” for this reason. As you can see, this choice also has the useful property that the measurement operator is unitary, and so may be identified with an operator, with the possible outcomes represented by the eigenvalues and eigenstates of that operator.

This mapping, from a unitary to a measurement, is possible to do for any operator that is both unitary and Hermitian, meaning that $U^{-1} = U^\dagger = U$. Such unitaries are easy to recognize: They are the ones with real values along the diagonal, and complex-conjugate symmetry over the off-diagonals: $U[i, j] = U[j, i]^*$.

For example, in addition to the Z and X , we also have a $Y := iXZ$. It turns out that any measurement that is possible to perform on a single qubit, can be written as a linear combination of these three matrices, which are known as the *Pauli matrices* after their discoverer.

Exercise 10. Show that interpreting the X (a.k.a. “NOT”) gate as a measurement operator leads to the possible outcomes $|+\rangle$ and $|-\rangle$ (where these are as defined in Exercise 4). What are the values of m_+ and m_- ?

Exercise 11. Write out the matrix representation of Y in the computational basis, and find the possible measurement outcomes it gives rise to when interpreted as a measurement operator by identifying its eigenstates and eigenvalues.

Measuring registers. We now give the last piece of the puzzle, namely a pseudocode for performing measurements on registers that may or may not be entangled with other parts of the system (i.e. may or may not be *pure*).

Definition 12 (Operating on registers). Let $\mathcal{Q}$ be an n -qubit quantum memory in quantum state $|\mathcal{Q}\rangle$, let the register $\mathcal{R}$ be an m -qubit subsystem of $\mathcal{Q}$, and let U be an $M \times M$ unitary matrix.

Next, define $U_{\mathcal{R}}$ to be the $N \times N$ unitary matrix that has the effect of applying U on the qubits in $\mathcal{R}$ and the identity on the qubits in $\mathcal{Q} \setminus \mathcal{R}$. Specifically:

$$U_{\mathcal{R}} := \text{SORT}^\dagger([\mathcal{R}], [\mathcal{Q} \setminus \mathcal{R}]) (U \otimes \mathbb{1}) \text{SORT}([\mathcal{R}], [\mathcal{Q} \setminus \mathcal{R}]),$$

where $\text{SORT}([\mathcal{R}], [\mathcal{Q} \setminus \mathcal{R}])$ is the $N \times N$ unitary matrix, composed of a series of SWAP gates, that reorders the qubits in $\mathcal{Q}$ such that the qubits of $\mathcal{R}$ precede the qubits of $\mathcal{Q} \setminus \mathcal{R}$; $\text{SORT}^\dagger$ reverses the operation of SORT ; and $\mathbb{1}$ is the $2^{n-m} \times 2^{n-m}$ identity matrix.

Then, the application of U to the qubits in register $\mathcal{R}$ is captured in the method $\text{Apply}_{\mathcal{Q}}(U : \mathcal{R})$, defined as

$$\text{Apply}_{\mathcal{Q}}(U : \mathcal{R}) : |\mathcal{Q}\rangle \longrightarrow U_{\mathcal{R}} |\mathcal{Q}\rangle.$$

²⁵ Historically, this choice corresponds to the possible outcomes of the *spin* observable for fermionic particles, such as electrons, with the value $+1$ corresponding to “spin-up” (the spin vector points in the positive direction of the z -coordinate) and the value -1 meaning “spin-down” (the spin vector points in the negative z -direction).

Definition 13 (Measuring a register). Let $\mathcal{Q}$ be an n -qubit quantum memory in pure state $|\mathcal{Q}\rangle$, let the register $\mathcal{R}$ be an m -qubit subsystem of $\mathcal{Q}$, and let $\mathbf{M}$ be an m -qubit projective measurement that check the state for membership in Hilbert subspaces $\{\mathcal{S}_i\}$.

Next, define $\mathbf{M}_{\mathcal{R}}$ to be the n -qubit projective measurement that has the effect of applying $\mathbf{M}$ on the qubits in $\mathcal{R}$ and the identity on the qubits in $\mathcal{Q} \setminus \mathcal{R}$ (constructed in the same manner as $\mathbf{U}_{\mathcal{R}}$ in Def. 12).

Then, performing measurement $\mathbf{M}$ on the qubits in register $\mathcal{R}$ is captured in the (probabilistic) method $\text{Meas}_{\mathcal{Q}}(\mathbf{M} : \mathcal{R})$ which observes $|\mathcal{Q}\rangle$ under $\mathbf{M}_{\mathcal{R}}$. Observing outcome i , designated by the measurement procedure returning value m_i , is written

$$i \leftarrow \$ \text{Meas}_{\mathcal{Q}}(\mathbf{M} : \mathcal{R}),$$

and the post-measurement will be

$$|\mathcal{Q}\rangle \xrightarrow{\mathbf{M}} |\mathcal{Q}\rangle_{\mathcal{S}_i},$$

where $|\mathcal{Q}\rangle_{\mathcal{S}_i}$ is the result of projecting the qubits of $\mathcal{R}$ onto subspace $\mathcal{S}_i$, and applying the identity to the rest.

We will omit the subscript whenever $\mathcal{Q}$ is clear from context, and the measurement operator $\mathbf{M}$ whenever we are performing a computational-basis measurement; for example, measuring ℓ qubits and observing the bit string $x \in \{0, 1\}^{\ell}$ will be written simply

$$x \leftarrow \$ \text{Meas}(\mathcal{R}).$$

To showcase the utility of our notation, we can now restate the Born rule in a very compact form.

Definition 14 (The Born rule). If qubit register $\mathcal{R}$ is in pure state $|\mathcal{R}\rangle = \sum_i \alpha_i |x_i\rangle$, then

$$\Pr[x \leftarrow \$ \text{Meas}(\mathcal{R}) : x = x_i] = |\alpha_i|^2.$$

A.8 Select Exercise Solutions

Solution to Exercise 1 Let U be such that, for all states $|\psi\rangle$, $\|U|\psi\rangle\| = \|\psi\|$. Using the definition of the norm, this gives us that $|(U|\psi\rangle)^\dagger U|\psi\rangle| = |\langle\psi|U^\dagger U|\psi\rangle| = |\langle\psi|\psi\rangle|$. Since this should hold for any $|\psi\rangle$, we must have that $U^\dagger U = \mathbf{1} = UU^\dagger$; the latter equality follows from the fact that, for all $|\varphi\rangle$ and $|\xi\rangle$, $|\langle\varphi|\xi\rangle| = |\langle\xi|\varphi\rangle|$. Thus, U satisfies the definition of an invertible matrix, with the inverse $U^\dagger$. Further, it is easy to see that the opposite also holds: any unitary matrix must be norm-preserving. Therefore, the set of norm-preserving operations on N -dimensional Hilbert space equal to the set of $N \times N$ unitary matrices.